

软件系统设计 系统架构

一、 软件系统架构简述

(一) 理解软件工程

软件与硬件的区别：

1. 软件是不可见的：软件是虚拟的，硬件是实体的。
2. 软件制作出来是为了被修改和改变的（软件的演化是其本质属性）

科学与工程的区别：

1. 科学的研究是研究这个世界的既有部分
2. 工程的研究是人类创造新的世界（不是因为人才产生的）

(二) 软件体系结构概念

Module，模块：是还没有实现出来的软件部分。

Component，组件：是已经实现出来的软件部分。

Connector，连接件：连接组件的部分

Element，元素：包含 Component 和 Connector。

(三) 体系结构在做什么？

1. 体系结构将系统分解成组件、模块和子系统，定义了组件接口（组件可以干什么）、组件交流和依赖（组件间如何沟通交流）、组件职责（当我们询问它时，组件能明确知道自己的职责），降低每一个部分的复杂度
2. 体系结构定义了通信的方式：数据通过机器传递，控制流传递。
3. 体系结构处理非功能需求：包括技术约束、业务约束和质量属性。

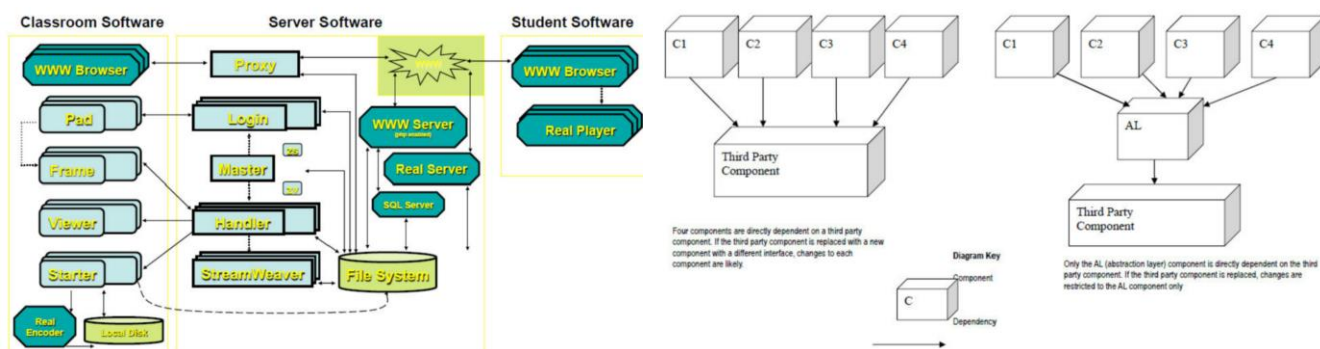
(四) 体系结构(Architecture) vs 设计(Design) vs 结构(Structure) vs 软件体系结构(Software Architecture)

1. 体系结构 (Architecture) 与设计 (Design):

- a) 体系结构是关于软件的，所有的体系结构都是软件设计，但不是所有的软件设计都是体系结构，体系结构是软件设计过程的一个过程。
- b) 其他观点：体系结构是更高层次的设计、是设计决策的组合、根据不同的项目而不同。

2. 体系结构 (Architecture) 与结构 (Structure):

- a) 体系结构是包含结构信息的。
- b) 结构是一种静态的、逻辑的、是关于系统如何构成的。
- c) 架构除了包含结构，还会包含组件之间的相关的关系结构，还会定义一些动态的行为（组件可能和谁记性交互）
- d) 如下左图是对结构的描述，右图是对体系结构的描述（其中左图第三方模块与其他四个模块直接耦合，而右图则加入 AL 模块来解耦）



3. 体系结构 (Architecture) 与软件体系结构 (Software Architecture)

- a) 体系结构的作用仍然保持一致：倾听用户理解需求、检查灵活性、形成实际结构版本创建蓝图、监督构建过程保证计划符合规范、引导暴风雨式的设计变更、危机和歧义。
- b) 软件体系结构是监督软件的构造过程：开发人员、工程师和设计者。

(五) 软件系统架构的来源

NFRs、ASRs、质量需求、涉众、组织、技术环境等等

(六) 如何创建一个设计

1. 设计活动目标：生成一个适应约束并且达到系统质量和业务目标的设计。

2. 广义设计策略

a) 分解：针对某一个系统关注点进行分解后处理。

- i. 质量属性可以分解，并分配给分解后的元素。
- ii. 如何对非 ASR 进行设计：ASR 仅意味着需求的优先级，仍然可以满足非 ASR 需求来满足其他人，除非您即将满足需求、需要重新确定需求优先级并重新设计和您不能满足需求。
- iii. 一次设计所有的 ASR 还是一个 ASR？经验问题。通过经验和教育，我们会有直观的设计方法，并使用模式/策略来设计多个 ASR。

b) 抽象：使用抽象让设计师关注本身结构而不是实现

c) 分而治之：对于一个大规模系统，每次聚焦一部分。

d) 生成与测试

- i. 将特定设计作为假设，当前假设的错误在下一次设计的假设中得到解决，而正确的谁设计得到保留。
- ii. 最初的假设来自于：现有系统、框架、模式与策略、设计清单（提供指导）
- iii. 下一个假设的生成：当前的假设、系统实现的具体情况和质量属性之间的差距、结合新选用的 Tactics。
- iv. 有哪些测试？根据分析技术、根据检查清单（职责分配、协作模型、数据模型、架构元素映射、资源管理、绑定时间、技术选择）

- v. 什么时候完成？具有满足 ASR 的设计、或者您将使用完预算进行设计时、实施出你做出的最佳假设时。

e) 迭代与细化：使用迭代的方法，每次对上一迭代进行精化。

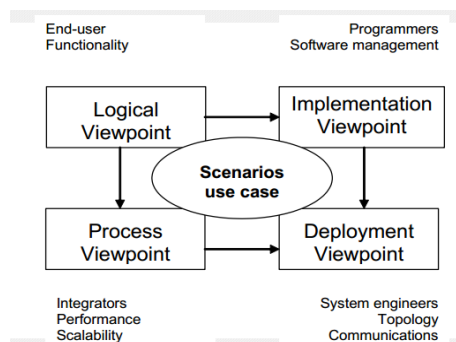
f) 重用元素：复用在设计过程中出现的可以复用的元素。

3. 体系结构视图

- a) 为什么使用体系结构视图？体系结构视图主要是为了应对软件不可见的问题，屏蔽其他没有影响的部分，将关注点分离。
- b) 什么是体系结构视图？一组系统元素和它们之间的关系表示（不是所有的系统元素，而是特定类型的元素）。
- c) 为什么软件系统架构需要用不同的视图来文档化？不同的视图支持不同的目标和用途，突出不同的系统元素和关系，在不同程度上暴露了不同的质量属性。

4. K.Kruchten 的 4+1 视图(绘图)

- a) 逻辑视图：描述了对架构而言重要的元素和他们之间的关系（功能需求）。
- b) 过程视图：描述了元素之间的并发和交互。
- c) 物理视图：描述了主要过程和组件是如何被映射到硬件上的。
- d) 发展视图：描述了软件组件的内部组织联系（比如使用配置管理工具存储）。
- e) 用例场景（Use Case）：捕获架构需求，与一个或多个特定视图相关。



（七）软件体系结构（Software Architecture）

1. 什么是？

- a) 是结构和系统结构，包含了软件元素，这些组件的外部可视化属性和他们之间的关系（包含组件的行为）。
- b) 定义 1：程序或计算系统的软件系统架构是系统的一个或多个结构，其中包括软件组件、这些组件的外部可见属性以及他们之间关系（SEI）。
- c) 定义 2：系统的基本组织体现在其组件、组件之间的相互关系、环境以及支配其设计和演进的原则（IEEE 1471-2000）

2. 活动：创建系统的商业案例、理解用户的需求、创建和选择体系结构、沟通体系结构（在涉众、开发商之间）、分析或评估体系结构（整体方法论和具体技术的质量）、实现体系结构、保证体系结构的一致性

3. 软件系统架构的角色：

- a) 代表决定如何实现需求的决策的**第一批人工制品**（早期设计决策），代表了**最难更改**的设计决策，值得最详细的考虑。
- b) 架构是完成产品生产线工程（**Product line Engineering**）的关键，与独立开发每个系统相比，以较少的工作量、成本和风险来一系列相似系统进行严格的开发。
- c) 当人开始在系统上工作时，构架通常是首先要检查的设计工件。
- d) 软件架构为**维护和修改**决策提供了参考框架

4. 为什么软件架构是重要的？

- a) 软件架构提供了沟通的工具：是可以确定和谈判利益冲突的参考框架（与用户

讨论需求、保证客户获取到过程和成本信息、实现决策和分配的管理)

b) 软件架构表现了最早期的决策集合

- i. 约束实现和开发者(实现需要符合架构,资源分配决策约束单独模块的实现)
- ii. 体现为开发和维护工作的组织结构:团队分工、预算和计划单位、基础工作拆分框架、文档的组织、CM库的组织、集成的基础、测试计划和测试基础、运维的基础。

c) 软件架构促进或阻碍质量属性的实现:灵活性、安全性、易用性等等

d) 软件架构会影响质量:但由于涉及许多其他因素,可能无法保证质量。

e) 软件架构引发有关潜在变更的讨论:软件架构将更改分为三种类型,本地(单组件修改)、非本地(多组件修改)、架构(修改系统基本结构、通信和协调结构)

f) 软件架构是一种可迁移和可重用的抽象:一对多映射

g) 软件架构是产品通用性的基础:整个产品线共享一个软件架构

h) 可以通过体系结构集成独立开发组件来开发系统(CBSE,基于组件的软件工程)

5. 软件架构过程

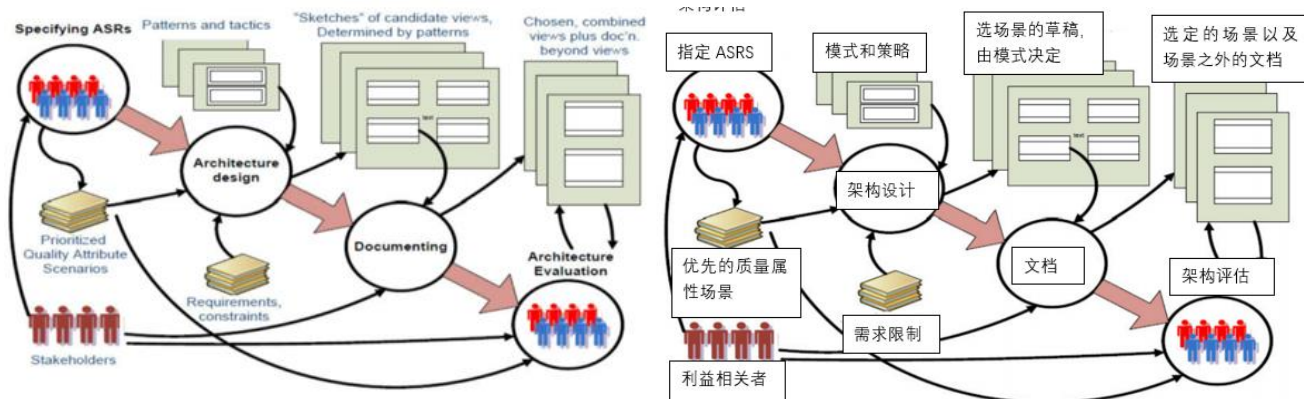
a) 通过 StackHolder 获取到 ASRs (架构攸关需求)

b) 通过分析得到 Prioritized Quality Attribute Scenarios(高优先级质量属性解决方案)和 Requirements, Constraints(需求和约束)

c) 将上述部分,结合模式和策略,综合可以得到架构的设计

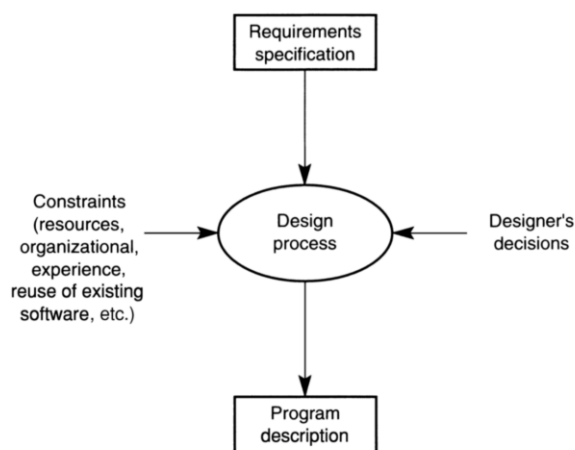
d) 根据架构的设计得到由模式决定的候选视图的示意图,之后完成文档化

e) 选择、组合视图,将文档进行进一步的评估,这一部分需要 StackHolder 的参与、也需要 Prioritized Quality Attribute Scenarios 和文档等作为参考。

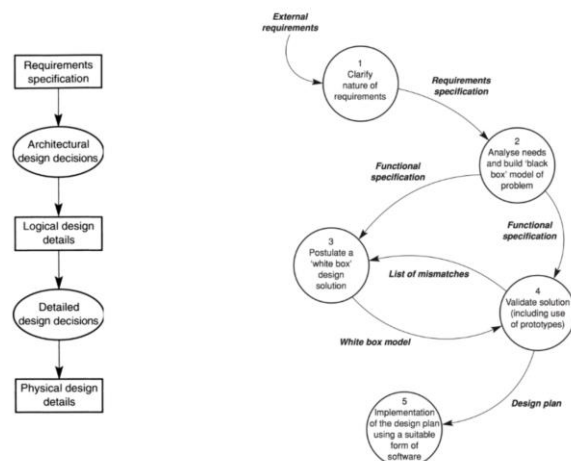


6. 部分过程图：

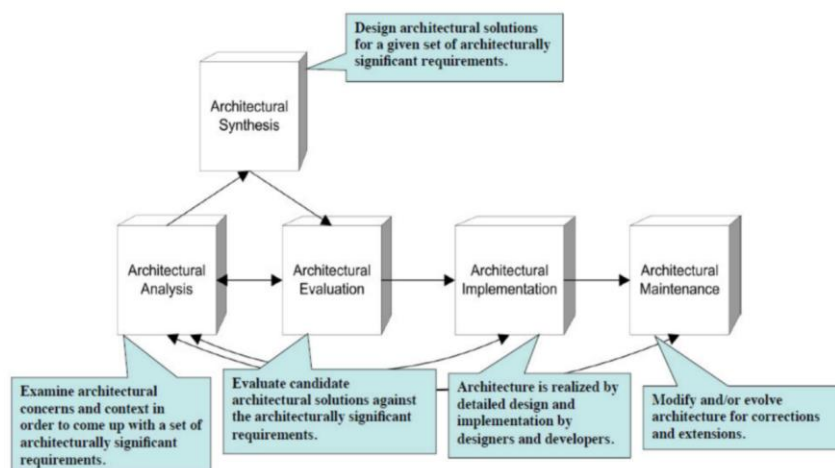
3.4. 总体设计模型 A General Design Model



3.5. 软件设计过程 Software Design Process

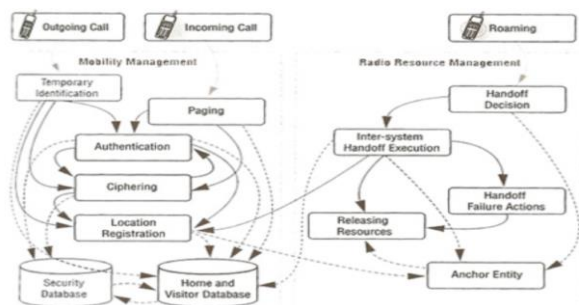


3.8. 体系结构生命周期 Architecture Lifecycle

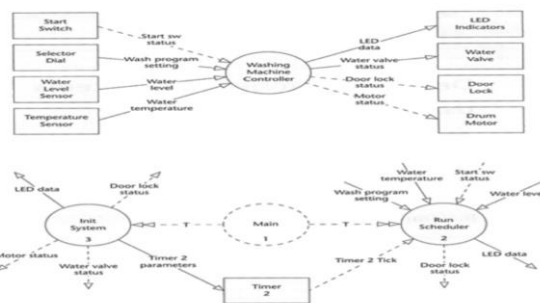


7. 软件架构实例

1.3.1. 移动手机系统架构 Mobile Phone System Architecture



1.3.2. 洗衣机架构 Washing Machine Architecture



8. 为什么要在架构中使用抽象？

软件体系结构提供了设计的更高层的抽象视角，更关注本身的结构而不是实现。

降低了架构设计时的系统复杂度，隐藏了设计的复杂性和实现，可能是或不是体系结构元素和软件元素之间的直接映射

(八) 架构知识域

软件系统设计基础概念

设计概念常识

上下文 (Context): 软件开发生命周期: 需求、设计、构造、测试

设计过程

在设计中使用的技术

关键点: 并发、控制和处理时间、分布式、异常处理、交互系统、持久化

软件结构和架构

架构的结构和关注点

架构的额模式和风格

软件设计方法: 架构方法 ADD、设计方法

二、 质量属性 Quality Attributes

(一) 需求

1. 功能性需求

- a) 定义系统必须做什么，并且强调系统如何提供价值给涉众。
- b) 功能是系统完成其预期工作的能力，很大程度上与结构无关。

2. 质量需求（替代术语：非功能需求、体系结构需求）

- a) 系统应在功能性需求之上提供的整个系统的合乎需求的特性。
- b) 质量属性是由软件系统的业务目标所决定的。
- c) 如果质量需求很重要，软件系统架构会将其映射到各种结构上。
- d) 分为两类
 - i. 执行过程中可观察（外部）：系统满足其行为要求的程度如何？例如性能、安全性、可用性等。
 - ii. 执行期间不可观察（内部）：系统的维护、集成或测试有多容易？例如可修改性、可移植性、可重用性和可测试性等。
- e) 为什么软件系统结构被认为是解决质量问题的最合适层次：
 - i. 开发完成后，质量不能被添加到软件密集型系统中，软件开发的所有阶段都需要解决质量问题：没有办法先实现功能，然后再尝试实现非功能性需求。
 - ii. 软件体系结构限制了各种质量属性的实现，例如性能、安全性和可用性等。

3. 质量属性

a) 确定质量属性

- i. 质量属性的精确定义对于在体系结构级别对其进行评估是必要的。
- ii. 使用质量属性方案用于定义所需的质量需求。

b) 质量属性有哪些: Adaptability 适应性、Extensibility 可扩展性、Availability 可用性、Modularity 模块化、Configurability 灵活性、Portability 可移植性、Flexibility 灵活性、Reusability 可重用性、Interoperability 互操作性、Testability 可测试性、Performance 性能、Auditability 可审核性、Reliability 可靠性、Maintainability 可维护性、Responsiveness 响应性、Manageability 可管理性、Recoverability 可恢复性、Sustainability 可持续性、Scalability 可扩展性、Supportability 可支持性、Stability 稳定性、Usability 可用性、Security 安全性。



（二）质量属性方案

1. 什么是质量属性方案

质量属性方案用于定义所需的质量属性，是具有一定结构的简单句子。方案的主要类别为通用方案与具体方案。

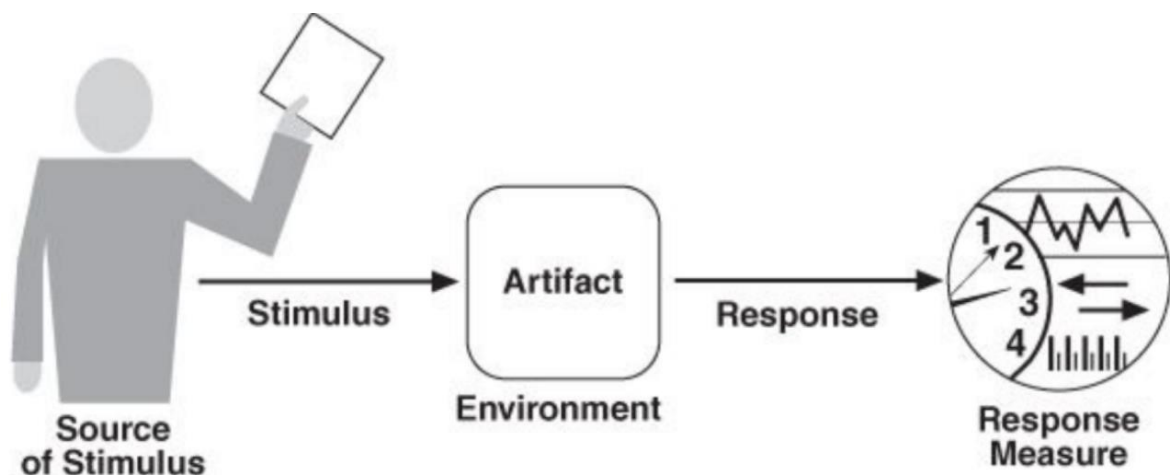
2. 通用方案

- a) **what?** 通用方案是与系统无关的方案，用于指导质量属性要求的规范。
- b) 它提供了一个框架，用于生成大量通用的、独立于系统的、质量属性特定的方案。
- c) 它包含的每种情况都可能但不一定与我们所关注的系统相关，为了使一般情况对特定系统有用，我们必须使他们特定于系统，即转化为特定系统的术语。

3. 具体方案

具体方案是系统特定方案，用于指导特定系统的质量属性要求的规范，是通用方案的实例。

4. 方案模板：表与 Stimulus-Response 图



- a) 刺激 (Stimulus): 到达系统时需要考虑的条件。
- b) 刺激源 (Source of Stimulus): 产生刺激的实体 (人, 系统或任何其他触发), 可能是输入、信息等, 对当前的状态的一个变化。
- c) 响应 (Response): 刺激到来后工件开展的行为。
- d) 响应度量 (Response Measure): 对刺激的响应以某种方法进行测量, 以便可以测试需求 (比如多长时间系统有反馈)
- e) 环境 (Environment): 发生刺激时系统的状况, 例如系统正常运行、系统过载、系统受到攻击、系统网络出现故障等。
- f) 工件 (Artifact): 完成需求的整个系统或者系统的一部分 (软件制品)。

5. 决策 (Tactics, 原子级别的最小的决定)

- a) Tactics 是影响质量属性相应控制的设计决策, 比如冗余。
- b) 体系结构策略: 决策的集合。
- c) 层次化决策: 决策也可由其他决策组成, 如冗余可划分为数据冗余和计算冗余。
- d) 样式 (Style) 或模式 (Pattern) 运用策略来提供预期的收益。
- e) 决策 (Tactics) 与模式 (Architecture Pattern)
 - i. 决策比模式更简单, 仅有单一的结构或机制来应对单一的架构驱动
 - ii. 决策是创建架构设计的重要组成块。
 - iii. 模式通常将多个设计决策组合到一个包中。
 - iv. 模式和策略共同构成了软件设计师的主要工具。
 - v. 大多数模式包含集中不同的策略, 这些策略可能有共同的目的或者经常被选择来实现不同的质量属性。

6. 质量设计决策

- a) 体系结构：设计决策的集合。
- b) 系统设计包括一组设计决策，其中一些决策可以用来保证质量属性响应，其他决策可以确保系统功能的实现。
- c) 七种设计决策（可能彼此覆盖）
 - i. 职责分配：将大的职责进行拆分。
 - ii. 协调模型：各部分之间的沟通与交互。
 - iii. 数据模型：数据格式、存储方式（缓存等）。
 - iv. 资源管理：CPU、网络、内存、时间等资源。
 - v. 架构元素映射：架构元素如何映射到具体的软件实现。
 - vi. 绑定时间决策：设置时间点（这个时间点前系统还是可以变化的，但之后不可以），比如选择安装环境需要在一个时间点前确认，同样的还有技术是否添加、编译时间、初始化时间、运行时绑定，但运行时弹性最大（尽可能延迟绑定时间）
 - vii. 技术选择：之前的部分已经确定后，我们可以选择的技术栈比较局限。

（三）约束

约束是具有**零自由度**的设计决策，是已经做出的预先指定的设计决策，需要通过接受设计决策并将其与其他受影响的设计决策进行协调，可以满足约束条件。

所有质量属性的 4+1 视图（Checklist）请参考 PPT。

(四) 可用性 Availability

可用性是应用程序的关键要求，以所需的可用时间来度量。

Outage、Failure、Fault 和 Error:

- 1. **Outage:** 系统不可用的情况，比如 Scheduled Downtime。
- 2. **Fault:** Failure 的原因。
- 3. **Error:** Fault 发生与 Failure 之间的中间状态。
- 4. **Failure:** 系统状态的可观察特征，系统无法交付该系统期望的服务。

影响因素：发现故障的时间、纠正故障的时间、重启应用的时间。

MTBF (Mean Time Between Failure)

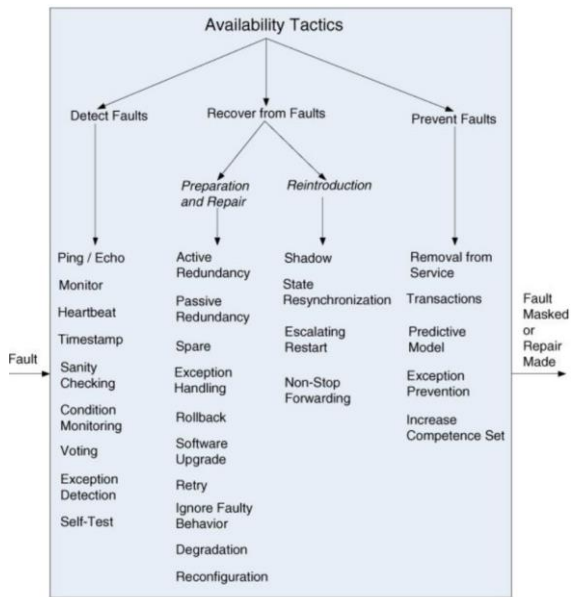
MTTR (Mean Time To Repair)

可用性: $MTBF / (MTBF + MTTR)$

1. 可用性通用方案

Portion of Scenario	Possible Values
Source	Internal/external: people, hardware, software, physical infrastructure, physical environment
Stimulus	Fault: omission, crash, incorrect timing, incorrect response
Artifact	Processors, communication channels, persistent storage, processes
Environment	Normal operation, startup, shutdown, repair mode, degraded operation, overloaded operation
Response	Prevent the fault from becoming a failure Detect the fault: - Log the fault - Notify appropriate entities (people or systems) Recover from the fault: - Disable source of events causing the fault - Be temporarily unavailable while repair is being effected - Fix or mask the fault/failure or contain the damage it causes - Operate in a degraded mode while repair is being effected
Response Measure	Time or time interval when the system must be available Availability percentage (e.g., 99.999%) Time to detect the fault Time to repair the fault Time or time interval in which system can be in degraded mode Proportion (e.g., 99%) or rate (e.g., up to 100 per second) of a certain class of faults that the system prevents, or handles without failing

2. 可用性策略



输入：Fault。

高可用性策略：消除单点故障、复制和故障转移、自动检测并重启。

输出：Fault 被屏蔽或完成恢复。

(五) 互操作性 (Interoperability)

互操作性是指两个或多个系统可以在特定的上下文中通过接口有效交换有意义的信息的程度，包括语法可操作性（交换数据的能力）和语义可操作性（能够正确解释数据）。互操作性需要确定谁，什么以及在什么情况下（上下文）。

影响因素：发现（发现服务位置、身份和接口）、处理响应（返回、转发、广播）。

1. 互操作性通用方案

Portion of Scenario	Possible Values
Source	A system initiates a request to interoperate with another system.
Stimulus	A request to exchange information among system(s).
Artifact	The systems that wish to interoperate.
Environment	System(s) wishing to interoperate are discovered at runtime or known prior to runtime.
Response	One or more of the following: ▪ The request is (appropriately) rejected and appropriate entities (people or systems) are notified. ▪ The request is (appropriately) accepted and information is exchanged successfully. ▪ The request is logged by one or more of the involved systems.
Response Measure	One or more of the following: ▪ Percentage of information exchanges correctly processed ▪ Percentage of information exchanges correctly rejected

2. 互操作性策略

输入：信息交换请求

定位：发现服务，通过已知目录服务来找到服务。

管理界面：编排（Orchestrate）、定制界面

输出：请求被正确处理

（六）可修改性 Modifiability

可修改性涉及到**更改**以及进行更改所需花费的**时间或金钱**，包括这种可变更性影响其他功能或质量属性的程度。

影响因素：变更是什么？变更可能性？何时谁进行变更？变更成本？

1. 可修改性通用方案

Portion of Scenario	Possible Values
Source	End user, developer, system administrator
Stimulus	A directive to add/delete/modify functionality, or change a quality attribute, capacity, or technology
Artifacts	Code, data, interfaces, components, resources, configurations, ...
Environment	Runtime, compile time, build time, initiation time, design time
Response	One or more of the following: ▪ Make modification ▪ Test modification ▪ Deploy modification
Response Measure	Cost in terms of the following: ▪ Number, size, complexity of affected artifacts ▪ Effort ▪ Calendar time ▪ Money (direct outlay or opportunity cost) ▪ Extent to which this modification affects other functions or quality attributes ▪ New defects introduced

2. 可修改性策略

输入：出现变更

拆分模块：将包含大量功能的模块拆分。

增加语义一致性：将用途不同的功能进行拆分。

减少耦合：封装（减少更改弥散的可能）、使用中介打破依赖、重构（两个模块受到相同变更的影响）。

延迟绑定：尽可能将绑定时间延迟（设计时、开发时、测试时、发布时）

输出：在一定时间和预算内实现变更。

（七）性能 Performance

性能与时间有关，和系统满足时序要求的能力有关（单位时间能做多少事情）。

影响要素：处理时间（正在相应）和阻塞时间（无法相应）。

1. 性能通用方案

Portion of Scenario	Possible Values
Source	Internal or external to the system
Stimulus	Arrival of a periodic, sporadic, or stochastic event
Artifact	System or one or more components in the system
Environment	Operational mode: normal, emergency, peak load, overload
Response	Process events, change level of service
Response Measure	Latency, deadline, throughput, jitter, miss rate

2. 性能策略

输入：事件到达

控制资源需求：管理采样频率、限制事件响应（排队）、事件优先级排序、减少过载、限制处理时间、提高资源利用率。

管理资源：增加资源、引入并发、计算资源多备份、数据资源多备份、限制队列长度、调度资源。

输出：在一定时间限制内响应生成。

（八）安全性 Security

安全性衡量系统保护数据和信息免遭未经授权应用的能力，同时仍提供对授权人员和系统的访问权限。

影响要素（CIA）：保密性（Confidentiality 防止未经授权访问）、完整性（Integrity 防止未经授权操作）、可用性（Availability 系统可供合法使用）。

1. 安全性通用方案

Portion of Scenario	Possible Values
Source	Human or another system which may have been previously identified (either correctly or incorrectly) or may be currently unknown. A human attacker may be from outside the organization or from inside the organization.
Stimulus	Unauthorized attempt is made to display data, change or delete data, access system services, change the system's behavior, or reduce availability.
Artifact	System services, data within the system, a component or resources of the system, data produced or consumed by the system
Environment	The system is either online or offline; either connected to or disconnected from a network; either behind a firewall or open to a network; fully operational, partially operational, or not operational.
Response	Transactions are carried out in a fashion such that - Data or services are protected from unauthorized access. - Data or services are not being manipulated without authorization. - Parties to a transaction are identified with assurance. - The parties to the transaction cannot repudiate their involvements. - The data, resources, and system services will be available for legitimate use. The system tracks activities within it by - Recording access or modification - Recording attempts to access data, resources, or services - Notifying appropriate entities (people or systems) when an apparent attack is occurring
Response Measure	One or more of the following: - How much of a system is compromised when a particular component or data value is compromised - How much time passed before an attack was detected - How many attacks were resisted - How long does it take to recover from a successful attack - How much data is vulnerable to a particular attack

2. 安全性策略

输入：攻击

检查攻击：发现入侵（通过流量或签名）、检测服务拒绝、检查消息完整性（通过校验或哈希）、检查消息延迟

防御攻击：验证（Identity）、授权（Authorize）、认证（Authenticate）请求者、限制资源访问、限制暴露、加密数据、分离实体、修改默认设置。

响应攻击：撤销对敏感资源访问、锁定电脑、告知请求者。

从攻击中恢复：对系统的追踪、快照存储。

输出：系统检测、防御、响应或恢复。

(九) 可测试性 Testability

可测试性是指可以使软件通过(通常是基于执行)测试来证明其故障的难易程度。

1. 可测试性通用方案

Portion of Scenario	Possible Values
Source	Unit testers, integration testers, system testers, acceptance testers, and users, either running tests manually or using automated testing tools
Stimulus	A set of tests is executed due to the completion of a coding increment such as a class layer or service, the completed integration of a subsystem, the complete implementation of the whole system, or the delivery of the system to the customer.
Environment	Design time, development time, compile time, integration time, deployment time, run time
Artifacts	The portion of the system being tested
Response	One or more of the following: execute test suite and capture results, capture activity that resulted in the fault, control and monitor the state of the system
Response Measure	One or more of the following: effort to find a fault or class of faults, effort to achieve a given percentage of state space coverage, probability of fault being revealed by the next test, time to perform tests, effort to detect faults, length of longest dependency chain in test, length of time to prepare test environment, reduction in risk exposure (size(loss) x prob(loss))

2. 可测试性策略

输入：执行测试。

控制和观察系统状态：专用界面、记录或回放故障、本地化状态存储、沙盒（保证实验消除后果）

限制复杂度：限制结构复杂性（减少依赖关系、继承深度、多态和动态调用）、限制行为不确定性。

输出：检测到 Faults。

(十) 易用性 Usability

易用性与用户完成所需任务的难易程度以及系统提供的用户支持的类型相关。

影响因素：学习系统功能、有效使用系统、最小化错误影响、使系统适应用户需求、增强信息和满意度。

1. 易用性通用方案

Portion of Scenario	Possible Values
Source	End user, possibly in a specialized role
Stimulus	End user tries to use a system efficiently, learn to use the system, minimize the impact of errors, adapt the system, or configure the system.
Environment	Runtime or configuration time
Artifacts	System or the specific portion of the system with which the user is interacting
Response	The system should either provide the user with the features needed or anticipate the user's needs.
Response Measure	One or more of the following: task time, number of errors, number of tasks accomplished, user satisfaction, gain of user knowledge, ratio of successful operations to total operations, or amount of time or data lost when an error occurs

2. 易用性策略

输入：用户请求。

支持用户操作：取消、撤销、暂停恢复、将对象成组操作

支持系统操作：维护任务模型（确定上下文对任务建模）、维护用户模型（预期的知识）、维护系统模型（预期的系统行为）。

输出：用户被给出正确的反馈和帮助。

（十一） 质量属性具体方案对比

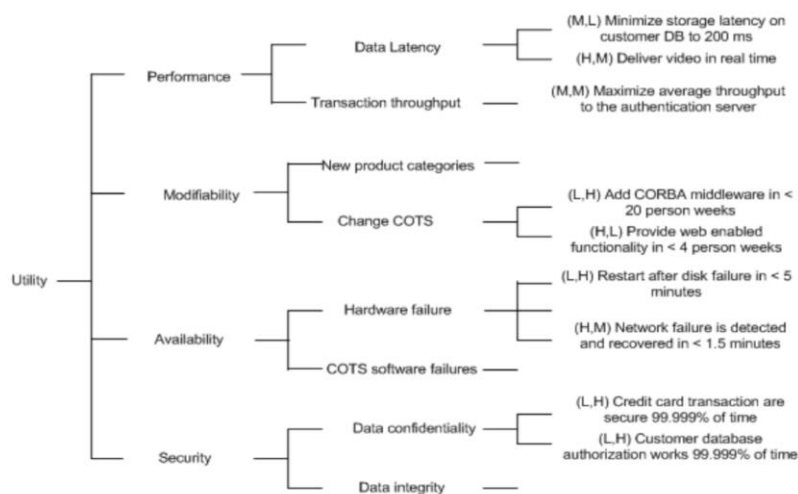
质量属性	刺激源	刺激	工件	环境	响应	响应度量
Availability	Heartbeat 监视器	服务器无响应	处理器	正常操作	通知操作者继续操作	没有停机时间
Interoperability	车辆信息系统	发送当前位置	路况监控系统	系统运行前已知	路况监控结合当前位置和 Google 地图上的其他信息，并且进行广播	我们的信息在 99.9%的时间是正确的
Modifiability	开发者	希望修改 UI 界面	代码	设计时	进行修改和单元测试	在 3 个小时内完成
Performance	用户	发起事务	系统	正常操作	事务被处理	平均延迟不超过 2 秒
Security	来自偏远地区心怀不满的员工	尝试修改支付率	系统内数据	正常操作	系统存储修改追踪	正确数据在 1 天内被存储并且进行篡改身份识别
Testability	单元测试者	单元测试完成	单元测试代码	开发时	记录结果	3 小时内达到 85%的路径覆盖
Usability	用户	下载新应用	系统	运行时	用户高效地使用应用	在 2 分钟以内的实验

（十二） 架构攸关的需求 Architecturally Significant Requirements, ASR

1. 什么是 **ASR**：架构攸关需求是对体系结构产生深远影响的需求（影响了关键体系结构设计决策）。
2. 需求越困难和重要，就越可能显著影响体系结构，从而成为 ASR。

3. 如何系统地识别 ASR 和其他因素：

- a) **从需求文档中收集 ASR：** 可以使用“MoSCoW”样式或用户故事来收集需求（不过难以收集质量需求），但其中大部分的内容都不会影响体系结构，对架构师有用的部分甚至没有出现在需求文档中。
- b) **通过采访涉众来收集 ASR：** 可以使用质量属性工作坊（QAW）
- i. QAW 演示和介绍
 - ii. 业务任务介绍
 - iii. 架构计划介绍
 - iv. 架构驱动程序的识别
 - v. 场景集思广益：每一个利益相关者表达一个场景
 - vi. 方案合并（合并类似方案）
 - vii. 方案优先级（通过投票）
 - viii. 方案细化
- c) **通过了解业务目标来收集 ASR**
- d) **通过质量属性实体树（Utility）来管理 ASR：** 使用方案量化描述需求后，逐渐对质量需求进行分解细化，直到含有量化指标为止。



三、 体系结构模式

(一) 领域特定的软件体系结构 **Domain-Specific Software Architecture**

领域特定的软件体系结构(DSSA)是软件组件的组合,专用于特定类型的任务(域),普遍使用于该领域的有效使用,可以有效构建成功应用程序的标准化结构(拓扑)组成。

DSSA 是最大限度地重用知识和进行先期开发并因此开发新的体系结构设计的卓越手段。

(二) 架构模式 **Architectural Patterns**

架构模式是在实践中反复发现的一套设计决策,具有允许重复使用的已知属性,并且描述了一类架构。

架构模式关联了如下三种角色:

1. **背景、上下文 (Context):** 世界上经常发生问题的场景。
2. **问题 (Problem):** 在给定上下文中出现经过适当概括的问题。
3. **解决方案 (Element + Relations + Constraints):** 针对问题的成功的经过适当抽象的解决方案。

(三) 分层模式 Layered Pattern 模块模式

分层模式用来构造可以分解为子任务组的程序，每个子任务组都处于一个特定的抽象级别，每层都为下一个提供更高层次的服务。分层模式的关键点在于确定依赖，核心是关注点分离（必须逐层访问）

上下文：一般桌面应用、Web（OSI 的七层网络模型）。

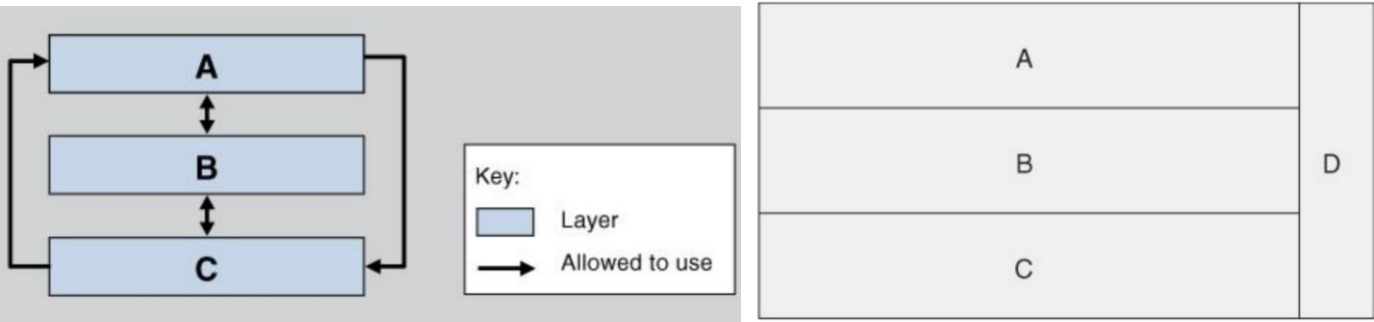
优点：高内聚、松耦合、易于维护

缺点：降低系统的性能，导致级联修改增加开发成本。

1. 分层模式的解决方案

Overview	The layered pattern defines layers (groupings of modules that offer a cohesive set of services) and a unidirectional <i>allowed-to-use</i> relation among the layers. The pattern is usually shown graphically by stacking boxes representing layers on top of each other.
Elements	<i>Layer</i> , a kind of module. The description of a layer should define what modules the layer contains and a characterization of the cohesive set of services that the layer provides.
Relations	<i>Allowed to use</i> , which is a specialization of a more generic <i>depends-on</i> relation. The design should define what the layer usage rules are (e.g., "a layer is allowed to use any lower layer" or "a layer is allowed to use only the layer immediately below it") and any allowable exceptions.
Constraints	▪ Every piece of software is allocated to exactly one layer. ▪ There are at least two layers (but usually there are three or more). ▪ The <i>allowed-to-use</i> relations should not be circular (i.e., a lower layer cannot use a layer above).
Weaknesses	▪ The addition of layers adds up-front cost and complexity to a system. ▪ Layers contribute a performance penalty.

2. 分层模式变体



左侧不是分层模式，形成环形依赖（没有实现关注点分离），是软件的坏味道。

右侧在 D 不期待 A 的结果且 D 不期待 B 的结果时是分层模式，但是在严格意义和其他场景中不是分层模式。

(四) 代理模式 Broker Pattern 组件-连接器模式

代理体系结构模式可以用于构建分布式软件系统，其中具有通过 RMI 交互的分离组件，代理组件负责协调通信，如转发请求，以及传输结果和异常。

上下文：多个同步或异步交互的远程对象组成的系统。

解决方案：通过提供隔离通讯相关的代理，将系统通信功能与主应用程序分开

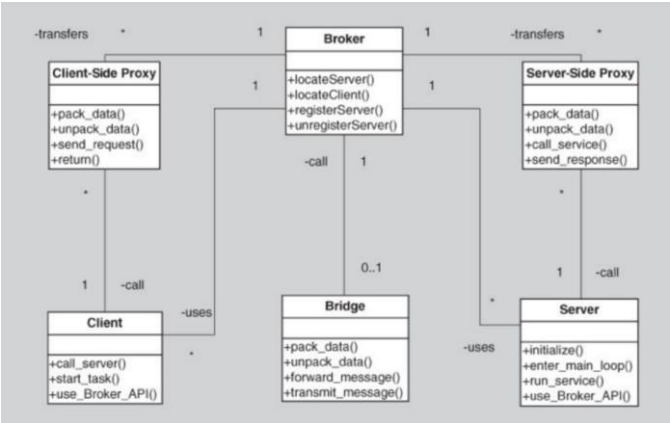
优点：提高了 Client 和 Server 之间的交互性，提高可伸缩性和可扩展性；整体大集群的性能可能会提高(QPS 等提高)，但是局部单点性能会下降，多次网络请求、多次匹配，有可能会抵消。

缺点：代理增加前期复杂度、可能成为通信的屏障、可能成为安全攻击的目标、难以测试。

1. 代理模式的解决方案

Overview	The broker pattern defines a runtime component, called a broker, that mediates the communication between a number of clients and servers.
Elements	<i>Client</i> , a requester of services <i>Server</i> , a provider of services <i>Broker</i> , an intermediary that locates an appropriate server to fulfill a client's request, forwards the request to the server, and returns the results to the client <i>Client-side proxy</i> , an intermediary that manages the actual communication with the broker, including marshaling, sending, and unmarshaling of messages <i>Server-side proxy</i> , an intermediary that manages the actual communication with the broker, including marshaling, sending, and unmarshaling of messages
Relations	The <i>attachment</i> relation associates clients (and, optionally, client-side proxies) and servers (and, optionally, server-side proxies) with brokers.
Constraints	The client can only attach to a broker (potentially via a client-side proxy). The server can only attach to a broker (potentially via a server-side proxy).
Weaknesses	Brokers add a layer of indirection, and hence latency, between clients and servers, and that layer may be a communication bottleneck. The broker can be a single point of failure. A broker adds up-front complexity. A broker may be a target for security attacks. A broker may be difficult to test.

2. 代理模式的概述图



(五) 模型-视图-控制器模式 MVC Pattern 组件-连接器模式

使用运行时、动态、相互之间的关系来审视，集成到了开发框架中，也是分层架构的变种（强调模块间约束关系，model 不可以直接返回到 controller），分为 model（业务逻辑）、view（处理用户展示，接收用户操作）、controller（对用户操作进行处理，将信息通知给 model）

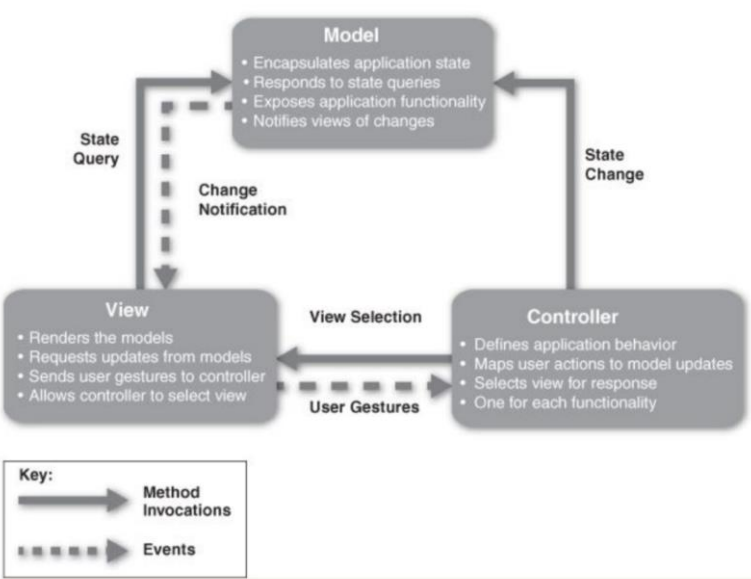
优点：耦合性低，重用性高，生命周期成本低，部署快，可维护性高，方便管理

缺点：没有明确定义，不适于中小型应用程序，增加实现复杂度，视图和控制器过于紧密，视图对模型访问低效。

1. MVC 模式的解决方案

Overview	The MVC pattern breaks system functionality into three components: a model, a view, and a controller that mediates between the model and the view.
Elements	The <i>model</i> is a representation of the application data or state, and it contains (or provides an interface to) application logic. The <i>view</i> is a user interface component that either produces a representation of the model for the user or allows for some form of user input, or both. The <i>controller</i> manages the interaction between the model and the view, translating user actions into changes to the model or changes to the view.
Relations	The <i>notifies</i> relation connects instances of model, view, and controller, notifying elements of relevant state changes.
Constraints	There must be at least one instance each of model, view, and controller. The model component should not interact directly with the controller.
Weaknesses	The complexity may not be worth it for simple user interfaces. The model, view, and controller abstractions may not be good fits for some user interface toolkits.

2. MVC 模式的概述图



(六) 管道和过滤模式 Pipe-and-Filter Pattern 组件-连接器模式

管道和过滤器模式应用在顺序处理结构中，有一系列 filter 体现依赖关系。

Filter: 相当于 Component，起数据处理、计算作用，每个 Filter 有 input 和多个 output，将数据处理后传递给后续部分。

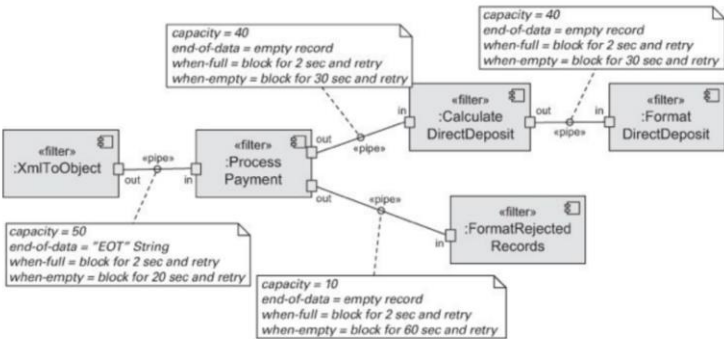
Pipe: 相当于 Connector，连接 filter，将 output 导入到其他的 filter 的 input 中去，不会独立存在。

缺点：不适用于互动式系统，过多的过滤器导致大量的计算开销。

1. 管道和过滤器模式的解决方案

Overview	Data is transformed from a system's external inputs to its external outputs through a series of transformations performed by its filters connected by pipes.
Elements	<i>Filter</i>, which is a component that transforms data read on its input port(s) to data written on its output port(s). Filters can execute concurrently with each other. Filters can incrementally transform data; that is, they can start producing output as soon as they start processing input. Important characteristics include processing rates, input/output data formats, and the transformation executed by the filter. <i>Pipe</i>, which is a connector that conveys data from a filter's output port(s) to another filter's input port(s). A pipe has a single source for its input and a single target for its output. A pipe preserves the sequence of data items, and it does not alter the data passing through. Important characteristics include buffer size, protocol of interaction, transmission speed, and format of the data that passes through a pipe.
Relations	The <i>attachment</i> relation associates the output of filters with the input of pipes and vice versa.
Constraints	Pipes connect filter output ports to filter input ports. Connected filters must agree on the type of data being passed along the connecting pipe. Specializations of the pattern may restrict the association of components to an acyclic graph or a linear sequence, sometimes called a pipeline. Other specializations may prescribe that components have certain named ports, such as the <i>stdin</i>, <i>stdout</i>, and <i>stderr</i> ports of UNIX filters.
Weaknesses	The pipe-and-filter pattern is typically not a good choice for an interactive system. Having large numbers of independent filters can add substantial amounts of computational overhead. Pipe-and-filter systems may not be appropriate for long-running computations.

2. 管道和过滤器模式的概述图



每一个组件表示 filter，连接组件的部分是 pipe，任何一个 filter 都依赖于前一个 filter 的输出（没有其他机会接收外部交互），不适合交互的场景（破坏依赖关系）。

(七) 客户端服务器模式 Client-Server Pattern 组件-连接器模式

客户端-服务器模型是一种分布式应用程序结构，它在资源或服务（称为服务器）和服务请求者（称为客户端）的提供者之间划分任务或工作负载。客户端和服务器通常通过计算机网络在单独的硬件上进行通信，但客户端和服务器可能位于同一系统中。服务器主机运行一个或多个与客户共享资源的服务器程序。客户端不共享其任何资源，但请求服务器的内容或服务功能。因此，客户端启动与等待传入请求的服务器的通信会话。使用客户端-服务器模型的计算机应用程序的示例是电子邮件，网络打印和万维网。

包含了两类不同的 Component, 没有 broker 可以动态改变 Client 和 Server 的关系，成对的关系相对固定。

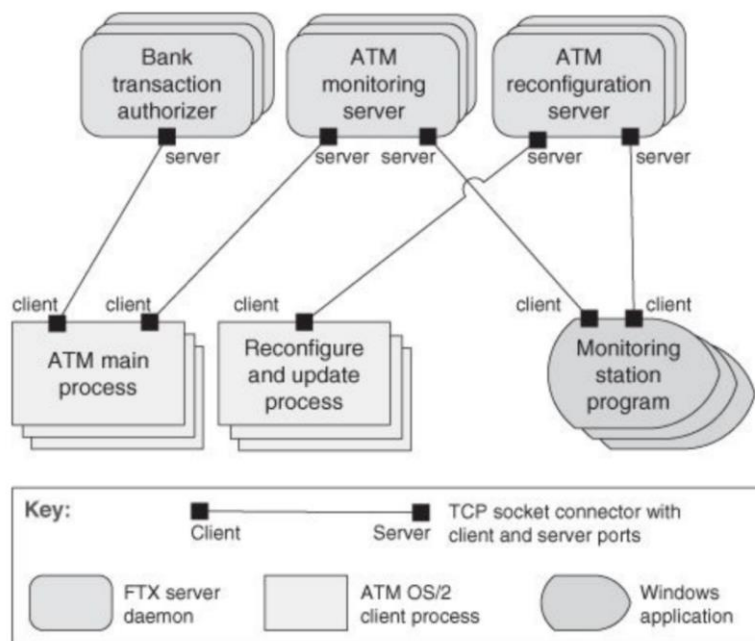
上下文：Windows 的客户端应用（桌面应用）。

缺点：服务器成为性能瓶颈、可能单点失效、决定在哪里实现功能的决定也是复杂的（并且难以修正。）

1. 客户端-服务器模式的解决方案

Overview	Clients initiate interactions with servers, invoking services as needed from those servers and waiting for the results of those requests.
Elements	<i>Client</i> , a component that invokes services of a server component. Clients have ports that describe the services they require. <i>Server</i> , a component that provides services to clients. Servers have ports that describe the services they provide. Important characteristics include information about the nature of the server ports (such as how many clients can connect) and performance characteristics (e.g., maximum rates of service invocation). <i>Request/reply connector</i> , a data connector employing a request/reply protocol, used by a client to invoke services on a server. Important characteristics include whether the calls are local or remote, and whether data is encrypted.
Relations	The <i>attachment</i> relation associates clients with servers.
Constraints	Clients are connected to servers through request/reply connectors. Server components can be clients to other servers. Specializations may impose restrictions: ▪ Numbers of attachments to a given port ▪ Allowed relations among servers Components may be arranged in tiers, which are logical groupings of related functionality or functionality that will share a host computing environment (covered more later in this chapter).
Weaknesses	Server can be a performance bottleneck. Server can be a single point of failure. Decisions about where to locate functionality (in the client or in the server) are often complex and costly to change after a system has been built.

2. 客户端-服务器端的概述图



ATM 验证身份，某一个验证 Server 可以给很多设备提供服务。

ATM 操作安全监控、盗刷之类，通过 monitoring 可以发现问题找到记录。

对于银行工作人员，需要添加新的业务，policy 发生变化，银行工作人员可以定义 ATM 的操作

银行负责安全金融的可以根据 ATM、对照用户操作行为是否有安全隐患，多对多。

3. 与 Broker Pattern 进行质量属性的影响方面的比较

Broker 也存在 Client 和 Server 之间的关系，但 CS 架构导致互操作性有所降低（没有 broker，需要人为制定固定连接），而 CS 不采用 broker 可能导致被拦截。

而在小型局域网（互联网还没有普及）时，规模有限，直接联系，性能与安全性平衡可能带来更大的收益。

(八) 点对点模式 Peer-to-Peer Pattern 组件-连接器模式

点对点模式中的组件，可能这一时刻为提供者，下一时刻是消费者（对等的）。同时点对点模式不仅仅提供服务，还能提供物流，每个 peer 可能有一个规定对的连接数。

对等（P2P）计算或网络是分布式应用程序体系结构，用于在对等体之间划分任务或工作负载。同行在申请中享有同样的特权，等同参与者。据说它们形成了节点的点对点网络。

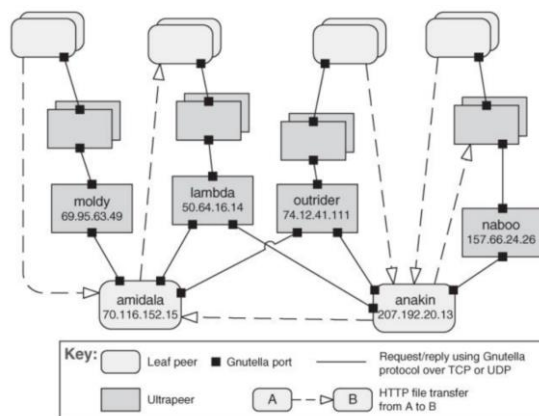
上下文：分布式应用程序体系结构，应用于对等体之间划分任务或工作负载。

缺点：安全性管理、数据持久化、数据/服务可用性（availability）、备份、修复更复杂、小型点对点系统不能持续实现质量目标，类似性能（performance）和可用性（availability）

1. 点对点模式的解决方案

Overview	Computation is achieved by cooperating peers that request service from and provide services to one another across a network.
Elements	<i>Peer</i> , which is an independent component running on a network node. Special peer components can provide routing, indexing, and peer search capability. <i>Request/reply connector</i> , which is used to connect to the peer network, search for other peers, and invoke services from other peers. In some cases, the need for a reply is done away with.
Relations	The relation associates peers with their connectors. Attachments may change at runtime.
Constraints	Restrictions may be placed on the following: ▪ The number of allowable attachments to any given peer ▪ The number of hops used for searching for a peer ▪ Which peers know about which other peers Some P2P networks are organized with star topologies, in which peers only connect to supernodes.
Weaknesses	Managing security, data consistency, data/service availability, backup, and recovery are all more complex. Small peer-to-peer systems may not be able to consistently achieve quality goals such as performance and availability.

2. 点对点模式的概述图



安全性：节点既是 Client 又是 Server，被攻击的可能性提高。

可用性：数据分布在不同节点上，相同数据多处拷贝，可能导致数据不一致，但是能保证个别数据出现问题不影响整体。

性能：多个节点同时提供服务，性能好（多个渠道获取数据，并行能力提高）

3. 客户端-服务器架构与点对点架构的比较

- 1、除了客户端-服务器模型之外，分布式计算应用程序通常使用对等（P2P）应用程序架构。
- 2、在客户端-服务器模型中，服务器通常设计为作为为许多客户端提供服务的集中式系统运行。服务器的计算能力，内存和存储要求必须适当地缩放到预期的工作负载（即同时连接的客户端数量）。负载均衡和故障转移系统通常用于扩展服务器实现。
- 3、在对等网络中，两个或多个计算机（对等体）汇集其资源并在分散的系统中进行通信。对等体是非分层网络中的同等节点或等效节点。与客户端-服务器或客户端-队列-客户端网络中的客户端不同，对等体直接相互通信。在对等网络中，对等通信协议中的算法平衡负载，即使是具有适度资源的对等体也可以帮助共享负载。如果节点变得不可用，则只要其他节点提供节点，其共

享资源就保持可用。理想情况下，对等体不需要实现高可用性，因为其他冗余对等体可以弥补任何资源停机时间；随着对等体的可用性和负载容量的变化，协议重新路由请求。

4、 客户端-服务器和主-从都被视为分布式对等系统的子类。

（九）面向服务的模式 Service-Oriented Pattern, SOA 组件-连接器模式

面向服务的体系结构（SOA）是一种软件设计风格，其服务通过应用程序组件，通过网络上的通信协议提供给其他组件。

面向服务的体系结构的基本原则独立于供应商，产品和技术。服务是一个独立的功能单元，可以远程访问并独立操作和更新，例如在线检索信用卡对帐单。

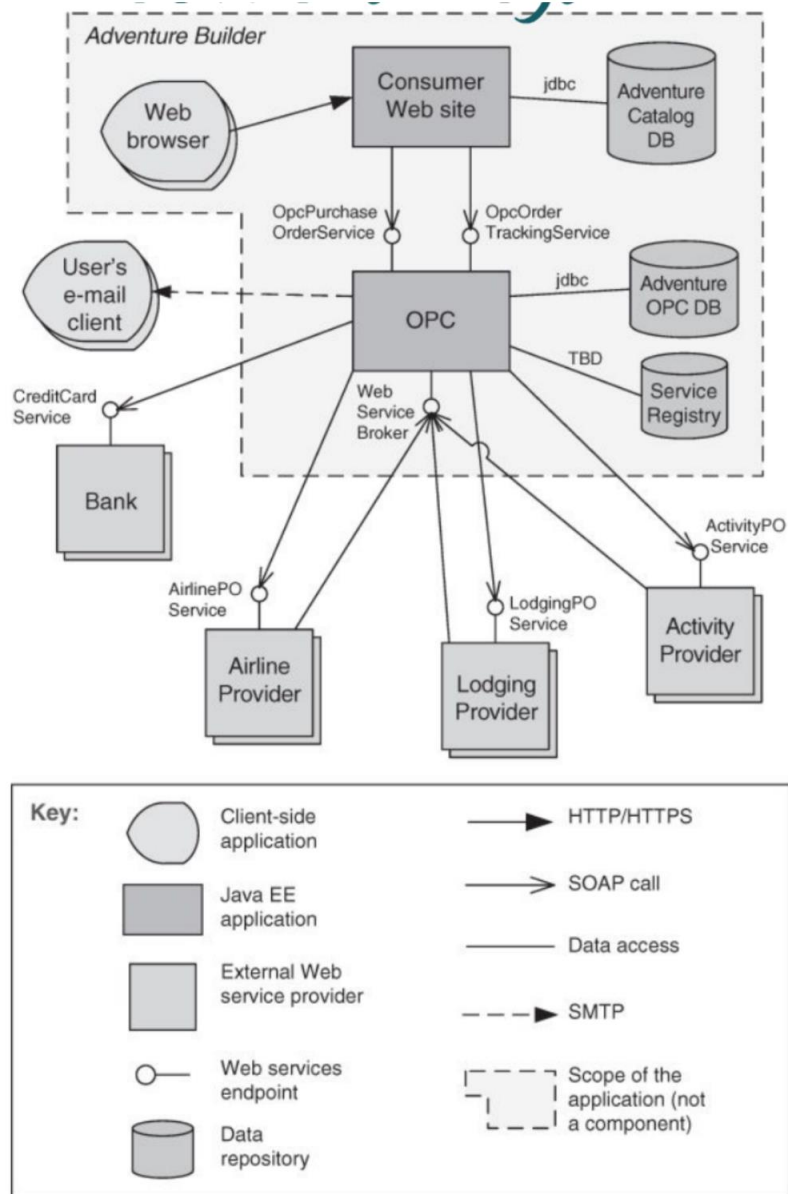
面向服务的模式是 **Broker Pattern** 的延伸，**Component** 包含服务提供者、服务消费者、**ESB**（企业服务总线）、企业服务组件、连接处理（注册、发现），**Connector** 包含 **SOAP**、**REST**、**Asynchronous messaging connector**。

缺点：构建复杂、无法管理独立服务的演化、服务可能成为性能瓶颈（无法提供性能保障）、使用中间件导致的性能开销（**Overhead**）。

1. 面向服务的模式的解决方案

Overview	Computation is achieved by a set of cooperating components that provide and/or consume services over a network. The computation is often described using a workflow language.
Elements	Components: • <i>Service providers</i>, which provide one or more services through published interfaces. Concerns are often tied to the chosen implementation technology, and include performance, authorization constraints, availability, and cost. In some cases these properties are specified in a service-level agreement. • <i>Service consumers</i>, which invoke services directly or through an intermediary. • <i>Service providers</i> may also be service consumers. • <i>ESB</i>, which is an intermediary element that can route and transform messages between service providers and consumers. • <i>Registry of services</i>, which may be used by providers to register their services and by consumers to discover services at runtime. • <i>Orchestration server</i>, which coordinates the interactions between service consumers and providers based on languages for business processes and workflows. Connectors: • <i>SOAP connector</i>, which uses the SOAP protocol for synchronous communication between web services, typically over HTTP. • <i>REST connector</i>, which relies on the basic request/reply operations of the HTTP protocol. • <i>Asynchronous messaging connector</i>, which uses a messaging system to offer point-to-point or publish-subscribe asynchronous message exchanges.
Relations	Attachment of the different kinds of components available to the respective connectors
Constraints	Service consumers are connected to service providers, but intermediary components (e.g., ESB, registry, orchestration server) may be used.
Weaknesses	SOA-based systems are typically complex to build. You don't control the evolution of independent services. There is a performance overhead associated with the middleware, and services may be performance bottlenecks, and typically do not provide performance guarantees.

2. 面向服务的模式的概述图



3. 面向服务的模式的连接器

- a) **SOAP (简单对象访问协议):** 服务提供者和提供者通过通常在 HTTP 之上交换请求/答复 XML 消息进行交互。
- b) **REST(代表性状态传输协议):** 服务使用者依赖于四个基本状态(POST、GET、PUT、DELETE) 的 HTTP 请求的 REST
- c) **异步消息传递 (即忘即发):** 参与者不必等待确认。

4. SOA 和其他架构的区别

- a) SOA 具备 Broker 的优势（而又不继承 Broker，因此 Broker 消失）。
- b) SOA 具有更高的互操作性和更高的伸缩性。
- c) 出现类似基础设施的组件（代替单一节点 broker），解决单点失效的问题，满足互联网普及、参与人数规模较大的问题。
- d) 商业模式的变化与技术可用性：技术条件下能够提供的服务越来越多，随着社会分工越来越细，服务也可以被拆分成更细、更大、更差异化、更细粒度，通过这种拆分可以提供差异化的服务，实现服务的动态绑定和替换；而微服务也是类似。

（十）发布-订阅模式 Publish-Subscribe Pattern 组件-连接器模式

Subscribe 对 Publish 进行注册，某个 Publisher 发布自己的消息可能订阅其他的信息（朋友圈微博）。随着订阅的增加，可能会导致性能的延迟（Subscriber 数量订阅越多性能下降）

在软件体系结构中，发布-订阅是一种消息模式，其中消息的发送者（称为发布者）不将消息直接发送到称为订阅者的特定接收者，而是将发布的消息送到连接器中（不知道有哪些订阅者）。类似地，订户表达对一个或多个类的兴趣并且仅接收感兴趣的消息，会接受到连接器的通知，而不知道哪些发布者（如果有的话）存在。

传统操作系统使用事件驱动方式来管理。

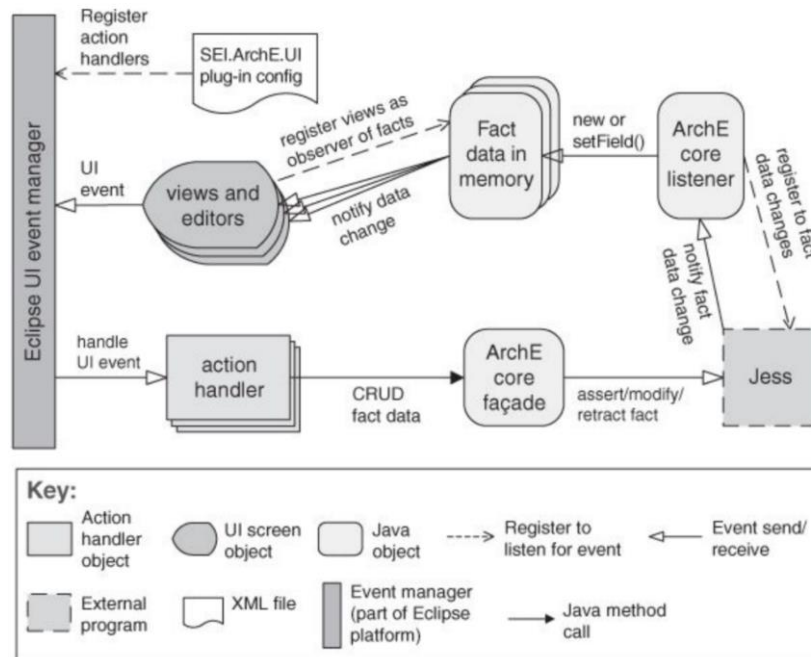
优点：松耦合、可伸缩性、此模式提供更高的网络可扩展性和更动态的网络拓扑，从而降低了修改发布者和已发布数据结构的灵活性。

缺点：通常会增加延迟并对消息传递时间的可伸缩性和可预测性产生负面影响。对消息排序的控制较少，并且不保证消息的传递。

1. 发布-订阅模式的解决方案

Overview	Components publish and subscribe to events. When an event is announced by a component, the connector infrastructure dispatches the event to all registered subscribers.
Elements	<i>Any C&C component</i> with at least one publish or subscribe port. Concerns include which events are published and subscribed to, and the granularity of events. <i>The publish-subscribe connector</i>, which will have <i>announce</i> and <i>listen</i> roles for components that wish to publish and subscribe to events.
Relations	The <i>attachment</i> relation associates components with the publish-subscribe connector by prescribing which components announce events and which components are registered to receive events.
Constraints	All components are connected to an event distributor that may be viewed as either a bus—connector—or a component. Publish ports are attached to announce roles and subscribe ports are attached to listen roles. Constraints may restrict which components can listen to which events, whether a component can listen to its own events, and how many publish-subscribe connectors can exist within a system. A component may be both a publisher and a subscriber, by having ports of both types.
Weaknesses	Typically increases latency and has a negative effect on scalability and predictability of message delivery time. Less control over ordering of messages, and delivery of messages is not guaranteed.

2. 发布-订阅模式的概述图



(十一) 共享数据模式 Shared-Data Pattern 组件-连接器模式

中间安全数据会被很多人共享登录访问。

一致性：CAP 原则（定理）：在一个分布式系统中，一致性（Consistency）、可用性（Availability）、分区容错性（Partition Tolerance），需要平衡三者。

一致性实现：最终一致性（不保证任何时间数据访问都一致但是保证最终结果要一

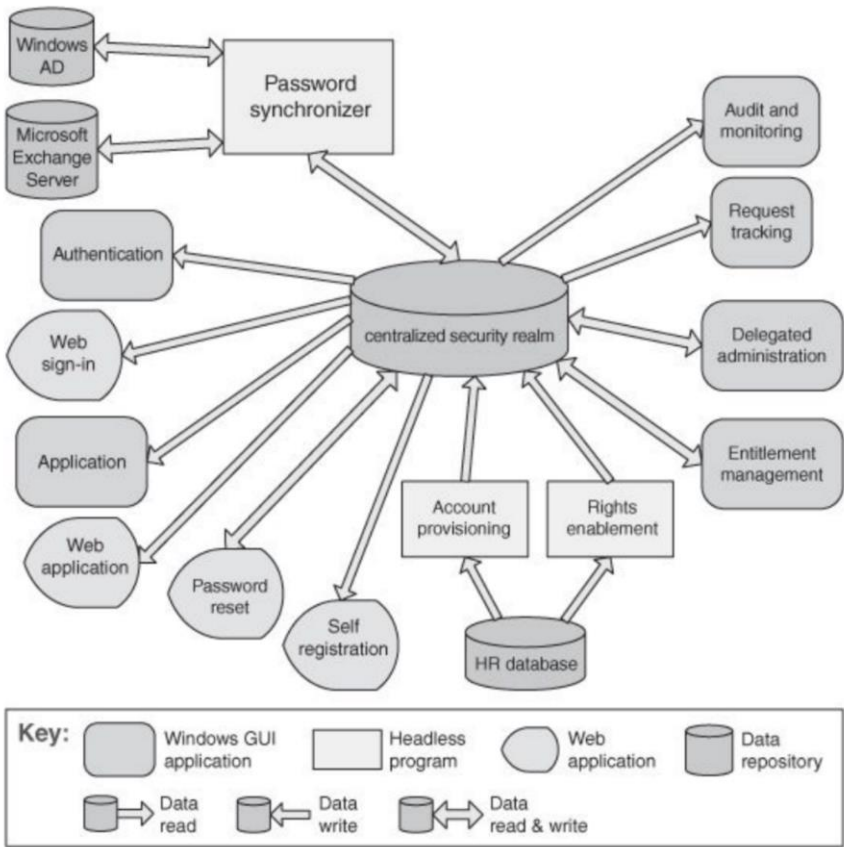
致)、互联网不可能牺牲可用性来保障一致性 (信号要求高), 因此设计难度非常大。

缺点: 共享数据模型可能存在性能瓶颈、可能存在单点失效、数据的生产者和消费者紧密耦合、中心点被攻击有安全风险、对安全性数据而言强调一致性, 如果副本有一段时间不一致可能会成为漏洞 (保证任何时刻访问数据都是一致的)。

1. 共享数据模式的解决方案

Overview	Communication between data accessors is mediated by a shared-data store. Control may be initiated by the data accessors or the data store. Data is made persistent by the data store.
Elements	<i>Shared-data store.</i> Concerns include types of data stored, data performance-oriented properties, data distribution, and number of accessors permitted. <i>Data accessor component.</i> <i>Data reading and writing connector.</i> An important choice here is whether the connector is transactional or not, as well as the read/write language, protocols, and semantics.
Relations	<i>Attachment relation</i> determines which data accessors are connected to which data stores.
Constraints	Data accessors interact with the data store(s).
Weaknesses	The shared-data store may be a performance bottleneck. The shared-data store may be a single point of failure. Producers and consumers of data may be tightly coupled.

2. 共享数据模式的概述图



(十二) Map-Reduce Pattern 分配模式

Map 负责对数据进行抽取所需的信息，完成信息转换。可以有很多个 Map，每个 Map 处理的数据工作内容不一样，他们互相独立运行。Reduce 负责进行合并，产出最终的答案。

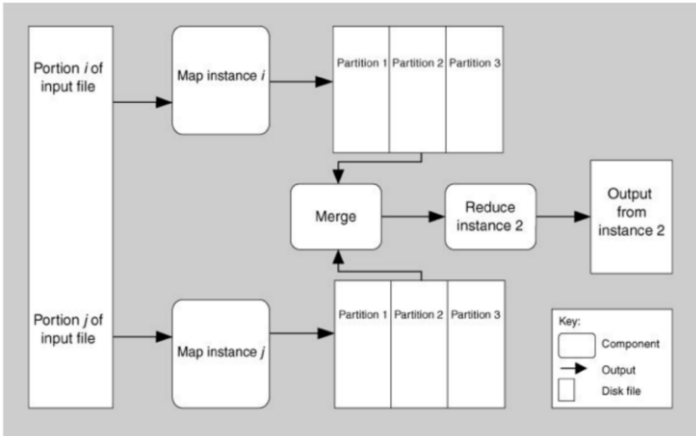
典型示例（词频统计）：每个 Partition 对应一个 Map，每个 Map 分别完成词汇词频的统计，最后有 Reduce 合并后进行排序。

缺点：如果没有足够大的数据集，那么 Map-Reduce 的开销是不能被接受的；如果不能将数据集切分成相似大小，那么并行的好处会消失；多次归纳是难以编排的。

1. Map-Reduce Pattern 的解决方案

Overview	The map-reduce pattern provides a framework for analyzing a large distributed set of data that will execute in parallel, on a set of processors. This parallelization allows for low latency and high availability. The map performs the <i>extract</i> and <i>transform</i> portions of the analysis and the reduce performs the <i>loading</i> of the results. (<i>Extract-transform-load</i> is sometimes used to describe the functions of the map and reduce.)
Elements	<i>Map</i> is a function with multiple instances deployed across multiple processors that performs the extract and transformation portions of the analysis. <i>Reduce</i> is a function that may be deployed as a single instance or as multiple instances across processors to perform the load portion of extract-transform-load. The <i>infrastructure</i> is the framework responsible for deploying map and reduce instances, shepherding the data between them, and detecting and recovering from failure.
Relations	<i>Deploy on</i> is the relation between an instance of a map or reduce function and the processor onto which it is installed. <i>Instantiate, monitor, and control</i> is the relation between the infrastructure and the instances of map and reduce.
Constraints	The data to be analyzed must exist as a set of files. The map functions are stateless and do not communicate with each other. The only communication between the map instances and the reduce instances is the data emitted from the map instances as <key, value> pairs.
Weaknesses	If you do not have large data sets, the overhead of map-reduce is not justified. If you cannot divide your data set into similar sized subsets, the advantages of parallelism are lost. Operations that require multiple reduces are complex to orchestrate.

2. Map-Reduce Pattern 的概述图



(十三) 多层模式 Multi-Tier Pattern 分配模式

Layer 是真实存在的，这里的层是逻辑的组合，没有层次模式的强依赖关系，在不同的部署环境中分层不同，但是软件完成内容一致。

许多系统的执行结构被组织成一组逻辑组件。每个分组被称为一个层。将组件分组到层中可能基于各种标准，例如组件的类型、共享相同的执行环境或具有相同的运行时目的。

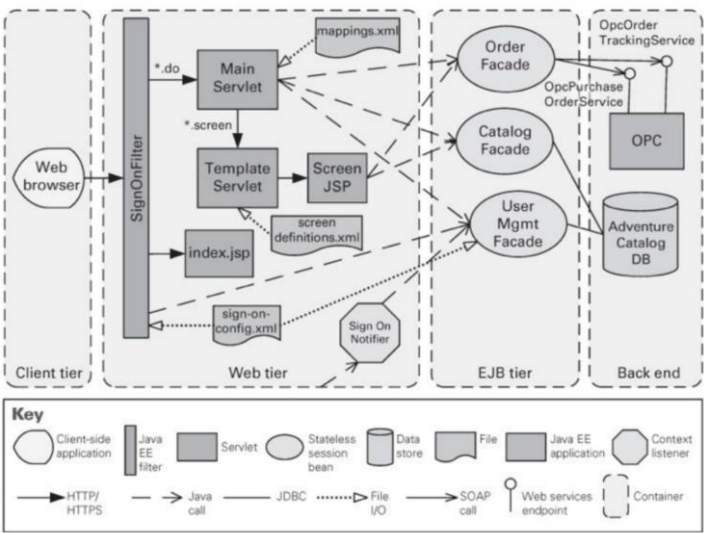
上下文：旅行社。

缺点：大量的前期成本和复杂性。

1. 多层模式的解决方案

Overview	The execution structures of many systems are organized as a set of logical groupings of components. Each grouping is termed a <i>tier</i> . The grouping of components into tiers may be based on a variety of criteria, such as the type of component, sharing the same execution environment, or having the same runtime purpose.
Elements	<i>Tier</i> , which is a logical grouping of software components. Tiers may be formed on the basis of common computing platforms, in which case those platforms are also elements of the pattern.
Relations	<i>Is part of</i> , to group components into tiers. <i>Communicates with</i> , to show how tiers and the components they contain interact with each other. <i>Allocated to</i> , in the case that tiers map to computing platforms.
Constraints	A software component belongs to exactly one tier.
Weaknesses	Substantial up-front cost and complexity.

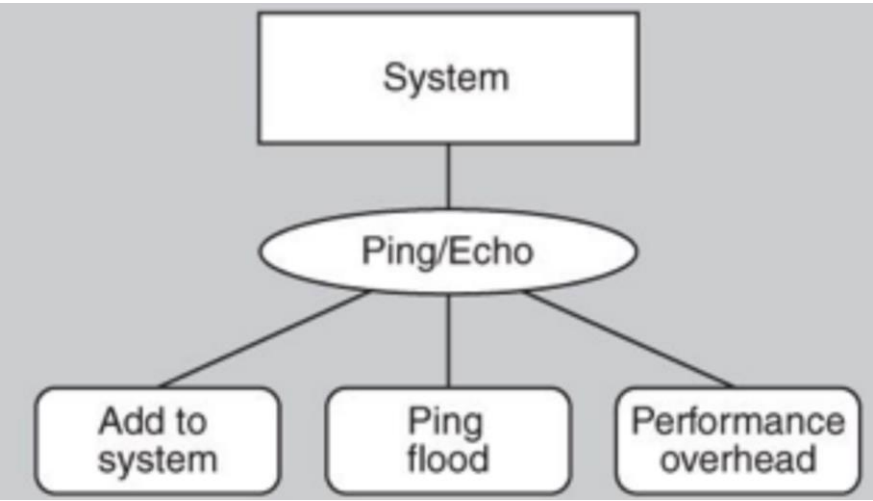
2. 多层模式的概述图



三大类：Modular 类、Runtime Process 动态类、软件和非软件环境关系（部署关系）

(十四) 架构设计与策略具体应用

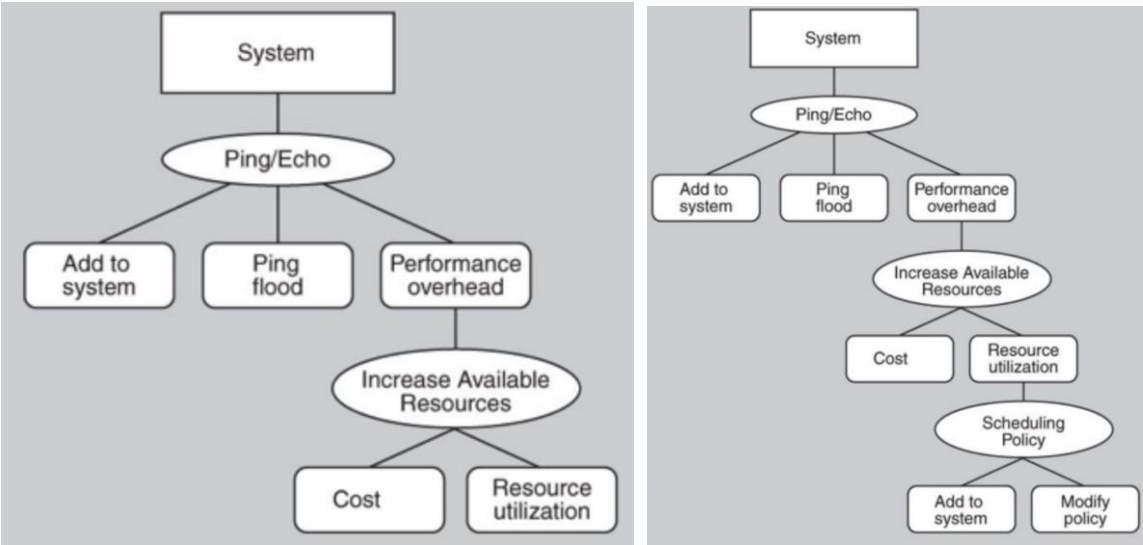
以 Layered Pattern 为例，目的是为了提高语义连贯性和限制依赖。系统设计全过程如下。首先目前系统如下所示：



椭圆形表示一个 Tactics，添加一个策略可能对某一个质量属性有贡献，但是会产生其他的负面影响（系统复杂度、资源影响、性能等）

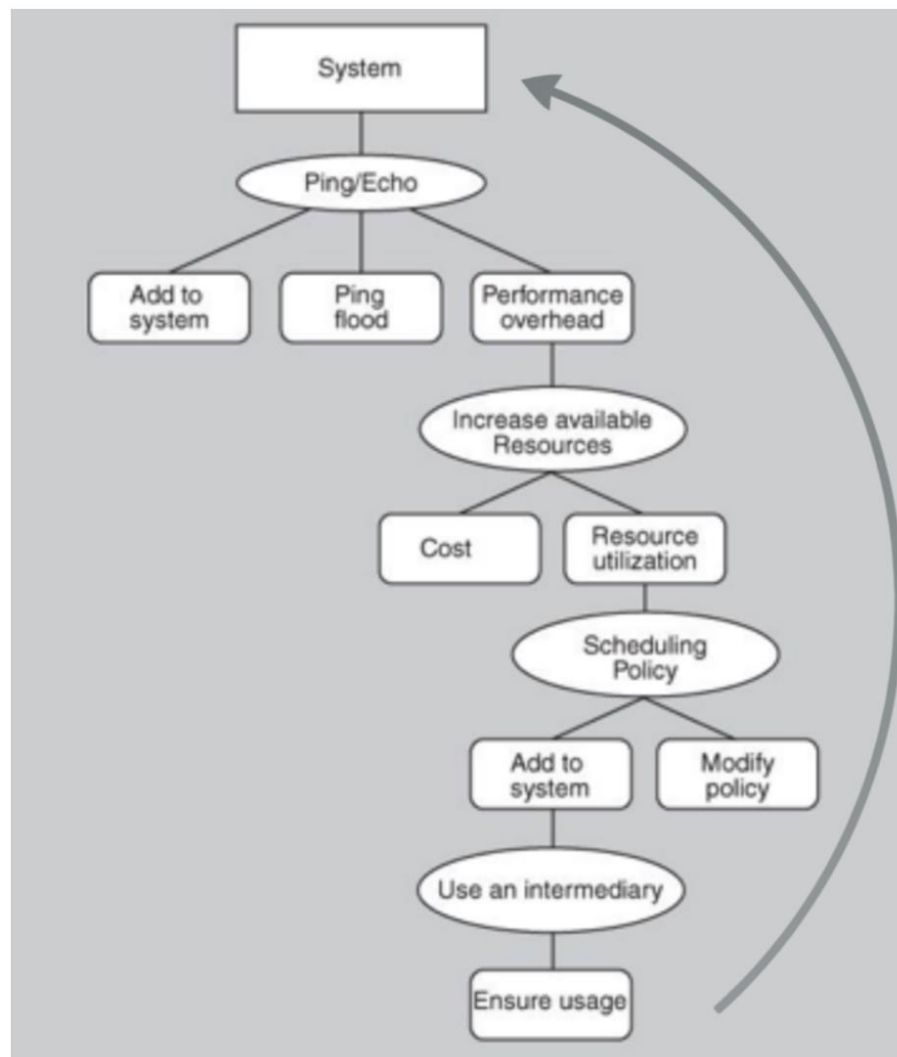
圆矩形表示添加策略后可能会导致的影响/后果。

假设对性能非常关注，那么我们进行单独的成本和资源利用率的设计考量如下：



为了提高资源的利用率，引入调度机制，而一切设计的可行性判断是根据 ASR 进行（分解到负面影响系统可以忍受则停止、没有足够人力物力则停止），以上整个过程汇不断迭代知道妥协（每一个 Tactics 的引入都会增加系统复杂度和规模）。

最终设计如下：



四、 属性驱动设计（Attribute-Driven Design, ADD）

（一） ADD 的输入和输出

1. ADD 的输入：需求

2. ADD 的输出：

- a) 软件元素：完成各种决策和职责、定义属性并与其他软件元素相关以组成系统架构的计算或开发工件。
- b) 角色：一组相关的职责。
- c) 职责：软件提供的功能、数据或信息。
- d) 属性：有关软件元素的附加信息。
- e) 关系：两个软件元素之间如何相互关联和交互的定义。

（二） 步骤 1：确定有足够的需求信息

系统涉众已经根据业务和任务目标确定了需求的优先级，同时您可以确定设计期间需要重点关注的系统元素。

确认是否有关于系统质量属性要求的足够关系：刺激-反应图（Stimulus-Response）

（三） 步骤 2：选择要分解的系统要素

如果是第一次设计未开发的系统时，需要将全部需求分配给系统。

如果是完善部分设计的系统时（系统已经被划分为多个元素，并分配需求），则从上述元素中选择一个元素作为关注点。

（四） 步骤 3：确定所选元素的 ASR

根据每个需求对涉众的重要程度（高、中或低），以及对架构的潜在影响（高、中或低）对所有的需求进行排名，得到（H,H）等二元组。

(五) 步骤 4: 选择符合 ASR 的设计

1. 步骤 4.1: 找出设计问题

如何解决设计中的 ASR? 如何将关注点划分为几个子关注点?

2. 步骤 4.2: 列出子关注点替代模式/策略

对于列表中的每个模式,都应该识别每个区分模式的区分参数(帮助在模式和策略中选择)、估计区分参数的值。

3. 步骤 4.3: 从清单中选择模式/策略

使用每种模式时有哪些权衡? 模式之间的结合程度如何? 是否有任何模式互斥?

4. 步骤 4.4: 确定模式/策略与 ASR 之间的关系

考虑到目前为止确定的模式/策略,并确定他们之间的关系(使用表格描述),所选的模式组合可能会构成新的模式。

5. 步骤 4.5: 记录初步的架构视图

通过开始记录不同的架构视图来描述模式,但不要求在此阶段创建完整记录的架构视图。

6. 步骤 4.6: 评估并解决不一致问题

根据架构驱动来评估设计的合理性

确定是否有架构驱动没有被考虑到

评估替代模式或者使用其他策略

将当前元素设计与体系结构中其他元素的设计进行评估,并解决所有的不一致。

(六) 步骤 5: 实例化架构元素并分配职责

实例化每一个所选的元素的实例

根据子元素类型来分配具体的职责

在子元素之间分配与父元素相关联的职责

分析并记录所做的设计决策

(七) 步骤 6: 为实例化元素定义接口

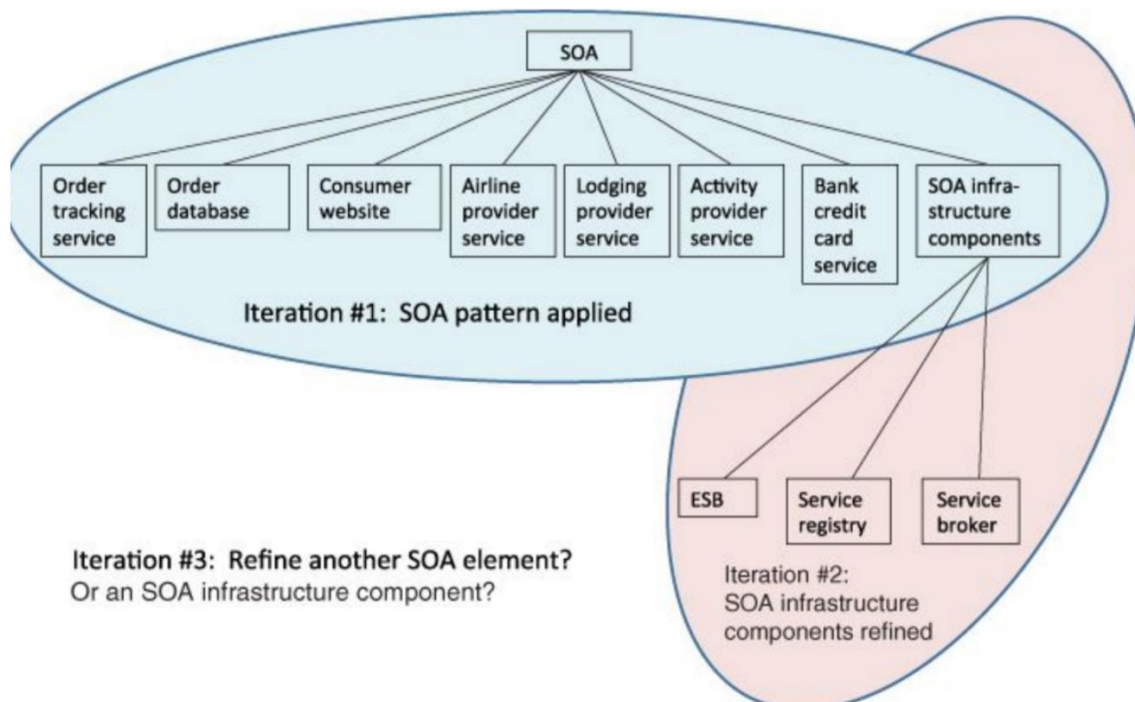
接口描述了提供（Provide）和需要（Requires）两种假设，即软件元素之间的互相联系，可以通过功能要求、观察信息传递消耗等方法。

(八) 步骤 7: 验证和完善需求

验证分配给父元素的所有需求是否被分配给一个或多个子元素。

将分配给子元素的所有的职责转换为各个元素的功能需求。

(九) 步骤 8: 重复进行，直到满足所有的 ASR



(十) 实例参考 PPT

五、 架构文档化

（一）为什么要记录软件架构

1. 用于交流和社交化架构设计决策。
2. 帮助理解和评估架构设计决策。
3. 刷新架构师对某些决策的记忆。
4. 培训架构设计人员。
5. 提供了维护和修改决策的框架。
6. 支持地理位置分散的团队。

（二）记录软件架构的挑战

1. 没有普遍接受的记录软件架构的标准和方法。
2. 记录大型系统的架构是一件耗时且重要的任务。
3. 对于用于记录架构视图的数量和性质没有达成共识，是资源密集型活动。
4. 迫在眉睫的最后期限和不断发展的架构性质不利于架构文档的流通。
5. 缺乏全面的符号和工具。

（三）软件架构文档记录的内容

1. 组件的接口和依赖项
2. 子系统的约束
3. 测试场景
4. 围绕设计决策的上下文信息

(四) 软件架构文档的记录内容的影响因素

1. 被记录的架构的复杂性
2. 应用程序的寿命
3. 基于涉众对文档的预期使用

(五) 架构文档化的原则

1. 从读者的角度撰写文档
2. 避免没有意义的重复
3. 避免模糊性
4. 使用标准的文档组织方法
5. 记录架构设计的理由
6. 保持文档最新但不要太新
7. 审查文档是否符合用途

(六) 体系结构文档的用途

1. 架构设计分析
2. 工作分解和分配
3. 部署后维护

(七) 视图和视图之外的部分

1. 视图 View

a) 架构风格(Style)分类

- i. 它是如何构建为一组实现单元的? Module Styles

- ii. 它是如何构建为一组运行时行为和交互的元素的？ **Component-Connector Styles**
 - iii. 它与环境中的非软件结构有何关系？ **Allocation Style**
- b) 架构风格（Style）与架构模式（Pattern）
- i. 架构风格
 - 1. 是元素和关系类型的特殊化，以及如何使用它们的一组约束。
 - 2. 侧重于架构方案，对特定风格何时有用或无用提供轻量指导。
 - ii. 架构模式
 - 1. 表达了软件系统的基本结构组织模式。
 - 2. 重要部分是关注问题和上下文，以及如何应用上下文解决问题。
 - 3. 架构模式：{问题，上下文} → 架构方法
 - iii. 模块视图 **Module View**（结构视图）：每个架构文档至少包含一个
 - 1. 模块是提供一组连贯职责的实现单元
 - 2. 视图示例：分解视图、使用视图、泛化视图、分层视图、领域视图、数据模型视图。

Elements	Modules, which are implementation units of software that provide a coherent set of responsibilities.
Relations	▪ <i>Is part of</i>, which defines a part/whole relationship between the submodule—the part—and the aggregate module—the whole. ▪ <i>Depends on</i>, which defines a dependency relationship between two modules. Specific module views elaborate what dependency is meant. ▪ <i>Is a</i>, which defines a generalization/specialization relationship between a more specific module—the child—and a more general module—the parent.
Constraints	Different module views may impose specific topological constraints, such as limitations on the visibility between modules.
Usage	▪ Blueprint for construction of the code ▪ Change-impact analysis ▪ Planning incremental development ▪ Requirements traceability analysis ▪ Communicating the functionality of a system and the structure of its code base ▪ Supporting the definition of work assignments, implementation schedules, and budget information ▪ Showing the structure of information that the system needs to manage

iv. C&C Views (Component-Connector Views) (结构视图)

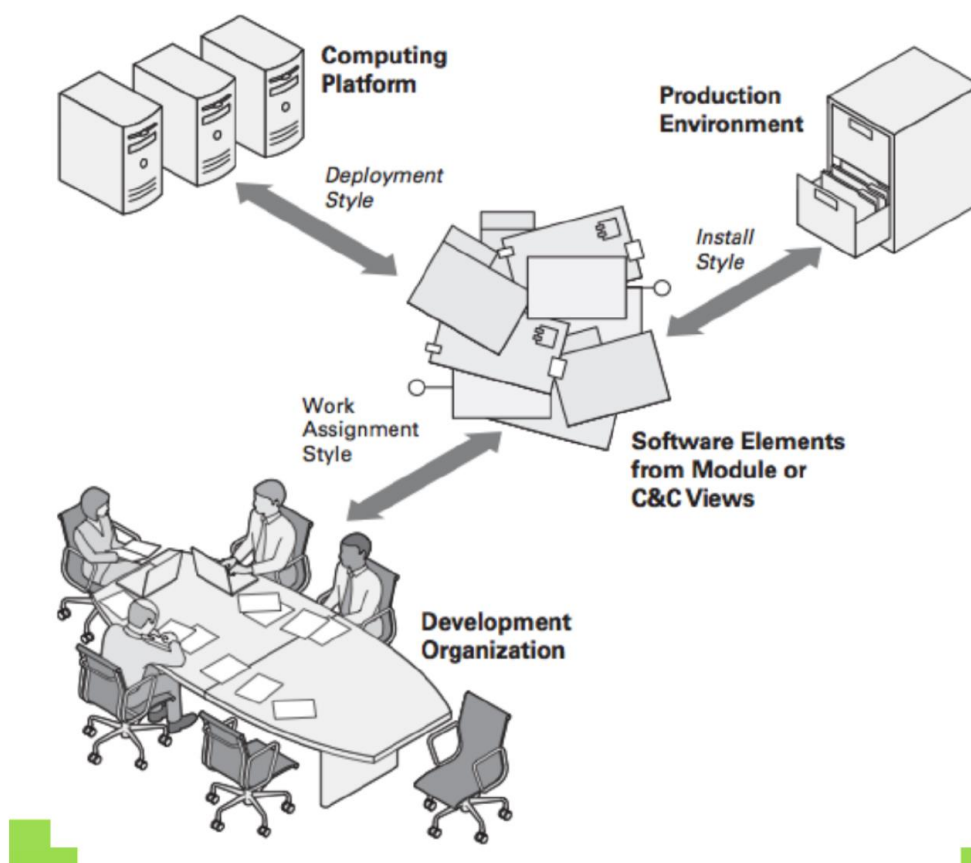
1. 显示具有某些运行时存在的元素，例如进程、对象、客户端、服务器和数据存储（组件）。
2. 附件知识有哪些连接器连接到哪些组件，通过将连接器的端点连接到组件的端口来显示附件。
3. 视图示例：管道和过滤器视图、客户端-服务端视图、点对点视图、面向服务的架构视图、发布-订阅视图

Elements	▪ <i>Components</i>. Principal processing units and data stores. A component has a set of <i>ports</i> through which it interacts with other components (via connectors). ▪ <i>Connectors</i>. Pathways of interaction between components. Connectors have a set of roles (interfaces) that indicate how components may use a connector in interactions.
Relations	▪ <i>Attachments</i>. Component ports are associated with connector roles to yield a graph of components and connectors. ▪ <i>Interface delegation</i>. In some situations component ports are associated with one or more ports in an “internal” subarchitecture. The case is similar for the roles of a connector.
Constraints	▪ Components can only be attached to connectors, not directly to other components. ▪ Connectors can only be attached to components, not directly to other connectors. ▪ Attachments can only be made between compatible ports and roles. ▪ Interface delegation can only be defined between two compatible ports (or two compatible roles). ▪ Connectors cannot appear in isolation; a connector must be attached to a component.
Usage	▪ Show how the system works. ▪ Guide development by specifying structure and behavior of runtime elements. ▪ Help reason about runtime system quality attributes, such as performance and availability.

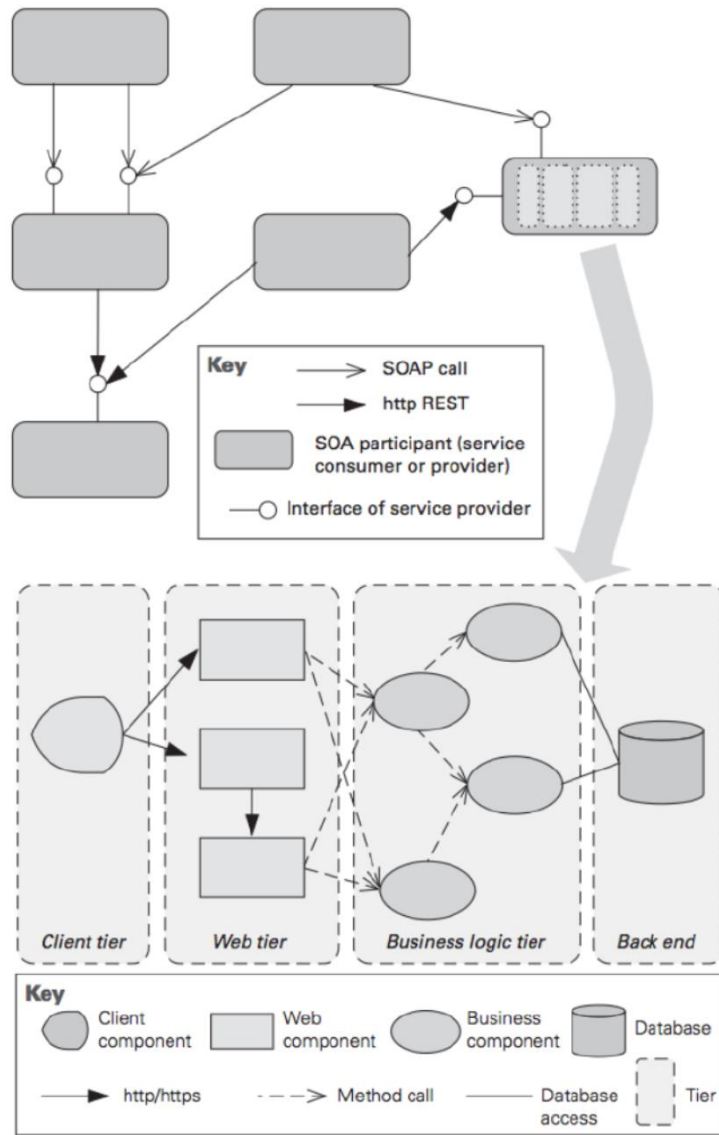
v. 分配视图 Allocation Views (结构视图)

1. 分配视图描述了软件单元到软件开发或执行环境元素的映射
2. 分配视图目标是将软件元素所需的属性与环境元素提供的属性比较，以确定分配是否成功。
3. 分配视图可以描述静态或动态视图。
4. 视图示例：部署视图、安装视图、工作分配视图、其他分配视图。

Elements	▪ <i>Software element</i>. A software element has properties that are <i>required</i> of the environment. ▪ <i>Environmental element</i>. An environmental element has properties that are <i>provided</i> to the software.
Relations	<i>Allocated to</i> . A software element is mapped (allocated to) an environmental element. Properties are dependent on the particular view.
Constraints	Varies by view
Usage	▪ For reasoning about performance, availability, security, and safety. ▪ For reasoning about distributed development and allocation of work to teams. ▪ For reasoning about concurrent access to software versions. ▪ For reasoning about the form and mechanisms of system installation.



- vi. 质量视图 **Quality Views**: 安全视图、性能视图、可靠性视图、沟通视图、异常（错误处理）视图
- vii. 组合视图 **Combining Views**:
1. 包括各种 C&C 视图、带有 SOA 或通信进程视图的部署视图、分解视图和任何工作分配、实施、使用或分层视图。
 2. 使用一张视图表示整个系统的部署信息，如下所示



viii. 如何选择视图

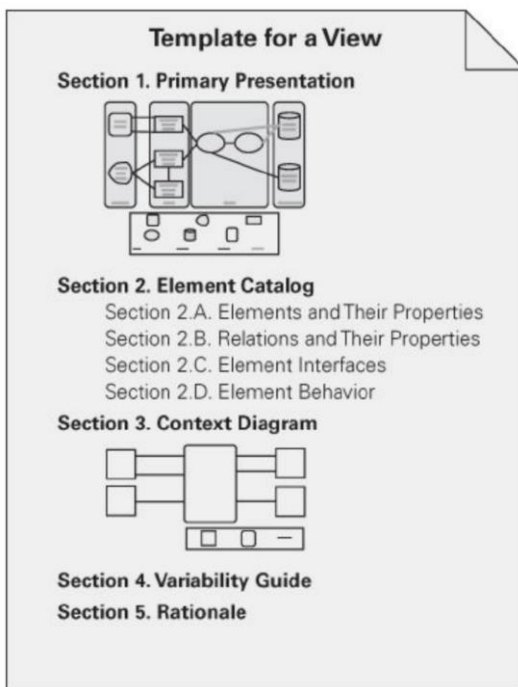
1. **构建涉众/视图表：**行为涉众，列为视图，格子填写涉众对视图的细节了解程度。
2. **合并视图**
 - a) 识别涉众/视图表中的边缘视图
 - b) 通过关联一个视图中的元素和另一个视图中的元素，将每个边缘视图与另一个具有更强表示能力的视图相结合。
3. **确定优先级和完成阶段**
 - a) 分解视图

b) 80/20 原则

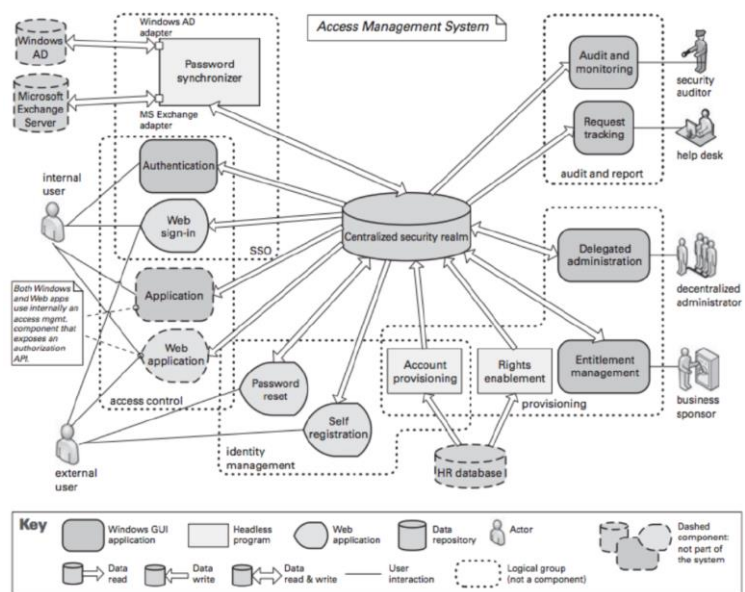
c) 按顺序完成所有视图

ix. 视图模板 View Template

1. 第一部分：主要介绍，显示视图的元素和关系，以及图例
2. 第二部分：元素介绍，详细介绍第一部分中描述的元素、元素属性、关系属性和元素接口和行为
3. 第三部分：上下文图，描述系统如何与环境相关
4. 第四部分：可变性指南，告知视图中可能发生的变化
5. 第五部分：基本原理，为什么设计反映在视图中，并且说明其合理性

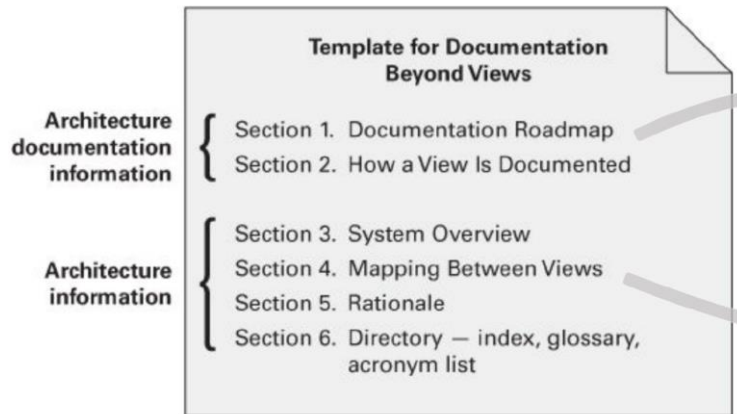


2.1.8. 上下文图 Context Diagram



2. 视图之外的部分 Beyond (Information Beyond Views)

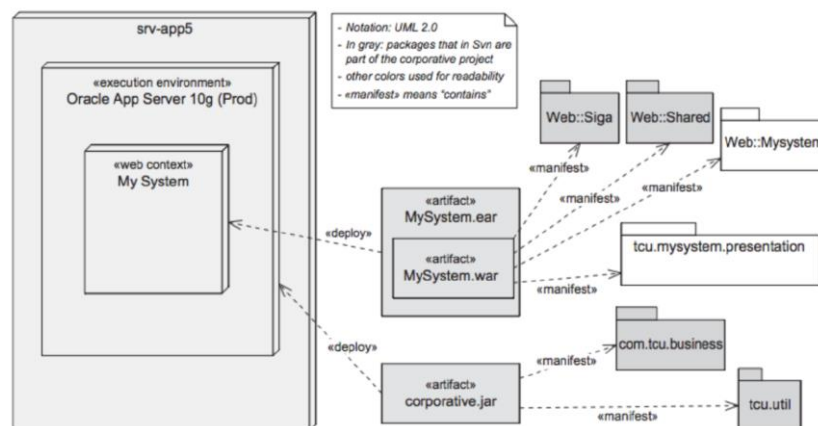
a) 这部分包含如下六个部分，此外还包含了文档控制信息 (Document Control Information)



b) 文档路线图 (Documentation Roadmap): 说明文档中的信息, 以及可以在哪里找到

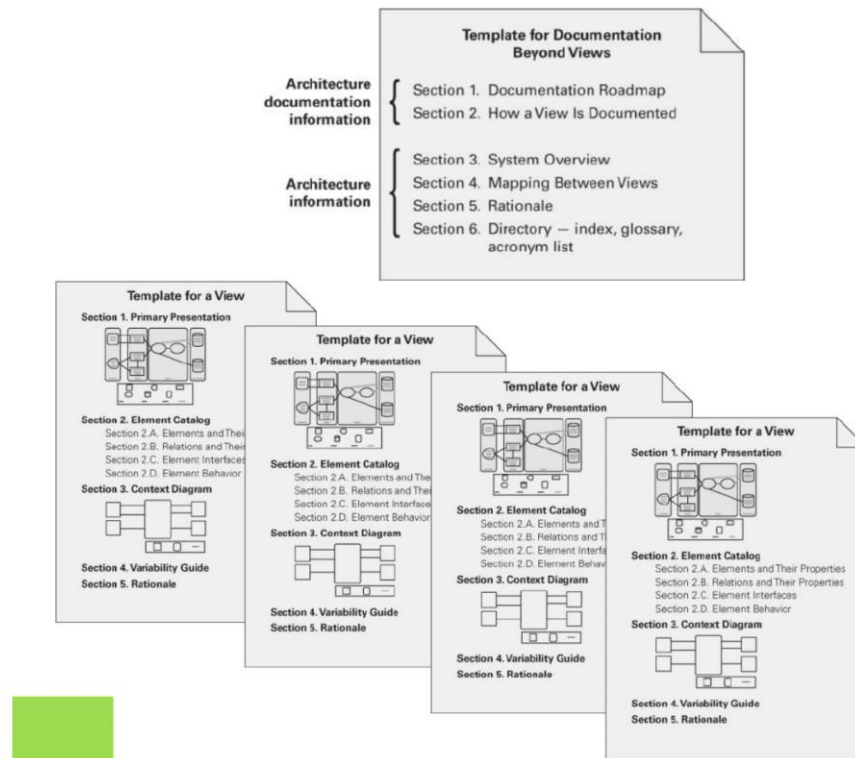
- i. 范围和总结
- ii. 文档的组织方式
 - 1. 简短的摘要
 - 2. 带注释的目录
- iii. 查看概览
- iv. 涉众如何使用本文档

c) 视图间映射 (Mapping Between View)



3. Documentation Package = Views + Beyond

2.2.4. Documentation Package



六、 软件体系架构评估

(一) 为什么进行软件体系架构评估

1. 大型项目经常出现延迟交付（设计失败）和超出预算（商业组件 CTOS 不能按期运行）的问题
2. 项目后期可能需要大量返工：团队没有考虑、缺少早发现问题的专家、有代价
3. 架构评估有助于缓解上述问题：彻底评估 CTOS 的稳定性、在问题修复成本较高前识别问题（设计缺陷、CTOS、组件的匹配性）、通知管理层以便其做出更好的决策。

（二）软件体系架构评估参与者

评估者可以是发现最佳实践的专家，且拥有不同项目的广阔视野。

设计过程中：

设计师：使用生成-测试方法

审查者：确定许多质量属性方案来推动检查

架构师：介绍要评估的架构部分

设计人员：遍历架构以解释如何满足场景

设计完成后：外部人员进行评估

（三）软件体系架构评估时间

1. 期望选择不合适的商业组件最小化风险时、涉及收购方和供应商时。
2. 设计时：新架构适合需求的早期“验证”
3. 演化和升级时：评估变化的影响
4. 构建时：实际架构是否按预期构建、是否被很好的构建
5. 尽早的进行评估，为什么？
 - a) 还有时间完成修复
 - b) 修复错误的决定成本还比较低
 - c) 是最有效的质量保证和风险缓解技术之一

（四）软件体系架构评估方法

软件体系架构评估方法需要有助于提出正确问题的方法

为什么软件体系架构评估方法有效：

1. 帮助涉众尽早提出正确的问题
 - a) 识别风险：发现可能对所需质量属性产生负面影响的架构决策。
 - b) 发现权衡：影响多个质量属性的架构决策。
 - c) 发现敏感点：特定质量属性对其敏感的架构决策。
2. 发现趋势：架构决策与系统属性预测之间的相关性。
3. 可以通过进一步分析、设计或原型制作来降低风险。
4. 所做的权衡和支持它们的理由可以适当地记录下来以备将来参考。

常见的基于场景的软件体系架构评估方法包括如下：

1. Software Architecture Analysis Method (SAAM)
2. Architecture Level Modifiability Analysis (ALMA)
3. Performance Assessment of Software Architecture (PASA)
4. Architecture Trade-off Analysis Method (ATAM)

（五）ATAM(Architecture Trade-off Analysis Method)：架构权衡分析方法

1. 阶段 0：准备和建立团队

参与者：评估团队领导和关键项目决策者

输入：架构设计文档

输出：评估计划

Who? 参与者

When? Where? How?

什么时候评估报告会被呈递给谁？

评估报告中应该包含什么信息？

2. 阶段 1：评估-1

参与者：评估团队和项目决策者

评估步骤：

- a) **第一步，介绍 ATAM：**评估负责人向项目代表介绍 ATAM，帮助其了解评估的过程和输出。
- b) **第二步，介绍业务驱动因素：**项目经理或客户从业务角度呈现系统概览，包括核心功能需求、技术管理经济或政治限制、商业目标和上下文、主要社会总、架构驱动因素（主要质量属性目标）
- c) **第三步，介绍体系架构：**
 - i. 首席架构师在适当细节级别上描述架构，包含技术限制（操作系统、硬件和中间件）、系统必须与之交互的其他系统、用于满足质量属性要求的架构方法。
 - ii. 架构演示（20 张 PPT，60 分钟），介绍架构驱动因素，相关度量数据以及满足这些要求的任何标准/模型/方法（2-3 页），重要的架构信息（4-8 页）包括上下文图、模块或层视图、组件和连接器视图、部署视图。采用的架构方法、模式或策略，包括它们解决的质量属性以及这些方法如何解决这些属性的描述（3-6 页）包括 CTOS 使用选择和集成、跟踪 1-3 个最重要的用例场景（描述资源消耗）、跟踪 1-3 个最重要的变换场景（描述变更影响）、与满足驱动架构要求相关的架构问题/风格、词汇表
- d) **第四步，确定架构方法：**ATAM 专注于理解架构方法来分析架构，评估团队通过研究架构文档、听取架构师的展示、向架构师询问设计系统时使用的模

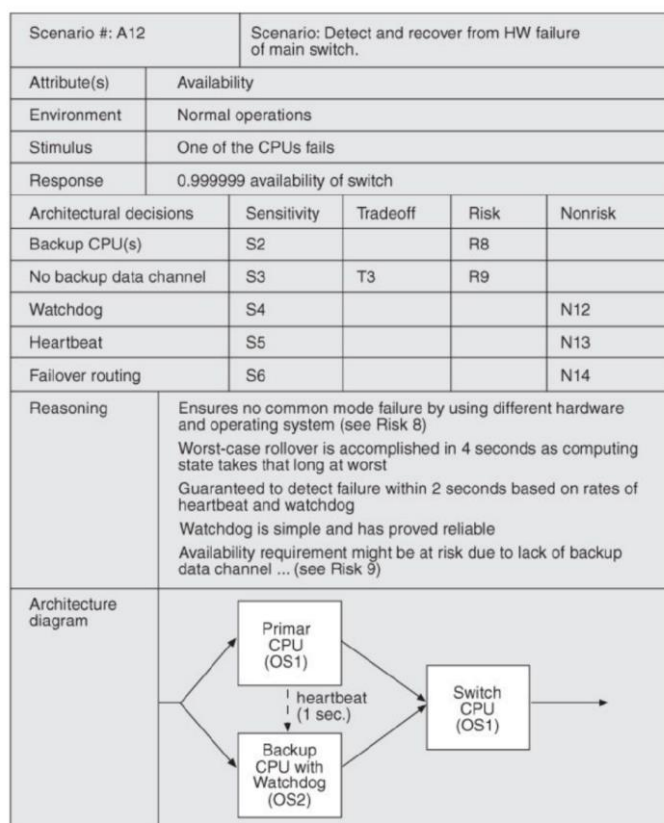
式和策略，完成对已确定的架构方法（风格、模式和策略）的编排。

e) **第五步，生成质量属性效用树（Utility Tree）：**是指导其余分析的关键步骤。

评估团队与项目决策者合作，确定、排序和优化系统最重要的质量属性目标，并最终用质量属性效用树来详细阐述，保证需求具体化

f) **第六步，分析架构方法：**

- i. 目标是让评估团队确信该方法的实例化适合满足特定于属性的要求，引出足够的架构信息，在已经做出的架构决策和需要满足的质量属性之间建立某种联系。
- ii. 评估团队听取架构师对架构互相支持的解释，检查质量属性效用树。
- iii. 评估团队记录相关的架构决策，并通过讨论识别和分类其中风险、非风险、敏感点和权衡。
- iv. 这一步结束后，评估团队应该清楚地了解整个架构的最重要方面、关键设计决策的基本原理以及风险、非风险、敏感点和权衡点的列表。



输出：

- a) 架构的简明介绍
- b) 业务目标（驱动因素）的说明
- c) 作为场景实现的特定质量属性要求的优先列表
- d) 效用树（Utility Tree）
- e) 风险和无风险部分
- f) 敏感点和权衡点

3. 阶段 2：评估-2

参与者：评估团队、项目决策者和架构涉众

评估步骤：

- a) 第一步，展示 ATAM 和之前已经取得的成果（除了质量属性效用树）
- b) 第七步，头脑风暴并确定场景的优先级：要求涉众集思广益，收集到有意义的场景并对其按优先级排序和合并，更大程度上了解涉众以及系统成功对涉众意味着什么。之后将优先从场景列表与使用程序树中的场景比较，如果差异比较大，那么额外的场景可能被识别为风险。
- c) 第八步，分析架构方法：该步骤和第六步相似，但针对新生成的场景中排名最高的（比如 5-10）
- d) 第九步，展示结果：评估团队根据共同的潜在问题或系统性缺陷将风险分组为风险主题，然后将风险主题域第二步中列出的特定业务驱动因素相关，最终将从评估收到的全部信息总结并呈现给所有涉众
 - 1. 记录的架构方法
 - 2. 集思广益的场景集以及优先级

3. 质量属性效用树
4. 发现的风险和非风险因素
5. 发现的敏感和权衡因素
6. 风险主题和受威胁的业务驱动因素

输出：

- a) 涉众们的优先场景列表
- b) 风险主题和业务驱动因素各自受到的威胁

4. 阶段 3：后续（Follow Up）

参与者：评估团队、主要涉众

评估过程：评估团队制作一份最终书面报告，分发给主要涉众以审核，审查结束后将报告提交给委托评估的人

输出：最终评估报告

Phase	Activity	Participants	Typical Duration
0	Partnership and preparation	Evaluation team leadership and key project decision makers	Proceeds informally as required, perhaps over a few weeks
1	Evaluation	Evaluation team and project decision makers	1–2 days followed by a hiatus of 1–3 weeks
2	Evaluation (continued)	Evaluation team, project decision makers, and stakeholders	2 days
3	Follow-up	Evaluation team and evaluation client	1 week

5. ATAM 方法的输出

- a) 架构的简明展示
- b) 业务目标的阐述
- c) 表现为质量属性场景的优先质量属性要求
- d) 质量属性效用树
- e) 风险和非风险组

- f) 风险主题组
- g) 将架构决策映射到质量要求
- h) 敏感和权衡组
- i) 最终的评估报告

6. 轻量级架构评估

Step	Time Allotted	Notes
1: Present the ATAM	0 hrs	The participants are familiar with the process. This step may be omitted.
2: Present Business Drivers	0.25 hrs	The participants are expected to understand the system and its business goals and their priorities. Fifteen minutes is allocated for a brief review to ensure that these are fresh in everyone's mind and that there are no surprises.
3: Present Architecture	0.5 hrs	Again, all participants are expected to be familiar with the system and so a brief overview of the architecture, using at least module and C&C views, is presented and 1 to 2 scenarios are traced through these views.
4: Identify Architectural Approaches	0.25 hrs	The architecture approaches for specific quality attribute concerns are identified by the architect. This may be done as a portion of step 3.
5: Generate Quality Attribute Utility Tree	Variable 0.5 hrs – 1.5 hrs	Scenarios might exist: part of previous evals, part of design, part of requirements elicitation. If you've got 'em, use 'em and make them into a tree. Half hour. Otherwise, it will take longer. A utility tree should already exist; the team reviews the existing tree and updates it, if needed, with new scenarios, new response goals, or new scenario priorities and risk assessments.
6: Analyze Architectural Approaches	2–3 hrs	This step—mapping the highly ranked scenarios onto the architecture—consumes the bulk of the time and can be expanded or contracted as needed.
7: Brainstorm and Prioritize Scenarios	0 hrs	This step can be omitted as the assembled (internal) stakeholders are expected to contribute scenarios expressing their concerns in step 5.
8: Analyze Architectural Approaches	0 hrs	This step is also omitted, since all analysis is done in step 6.
9: Present Results	0.5 hrs	At the end of an evaluation, the team reviews the existing and newly discovered risks, non-risks, sensitivities, and tradeoffs and discusses whether any new risk themes have arisen.
TOTAL	4–6 hrs	

七、 微服务架构

(一) 软件架构演化

1. 单体（Monolithic）架构

单体应用的全部功能被集成在一起作为一个统一的单元，更多作为应用部署架构。

2. 分层架构

核心是关注点分离原则，软件系统的组件被分为多个相互不重叠的层次，每一层都有特定的职能，仅处理本层的逻辑，而并不关心其他层的实现。

优点：结构简单、易于组织、便于独立测试维护

缺点：不易于实现持续发布部署、性能代价、可扩展性差

3. 面向服务架构（SOA 架构）

面向服务架构是一个分布式组件的集合，这些组件为其他组件提供服务（Provider）或者消费其他组件提供的服务（Consumer），而无需知道其他组件的细节。

企业服务总线（ESB）：为服务间相互调用提供支持环境，路由服务间信息，并对信息和数据进行必要转换。

服务编排引擎（Orchestration Engine）：可以根据预定义的脚本对服务消费者与服务提供者之间交互进行指挥。

优点：服务自身高内聚、服务间松耦合，最小化维护影响、良好的互操作性符合开放标准、模块化高可用性、服务的动态识别、注册、调用。

缺点：系统复杂性提高、难以测试验证、各独立服务的演化不可控、中间件容易成为性能瓶颈。

面向服务架构（SOA）实现原则：

服务契约:服务按照描述文档所定义的服务契约行事

服务封装:除了服务契约所描述内容，服务将对外部隐藏实现逻辑

服务重用:将逻辑分布在不同的服务中，以提高服务的重用性

服务组合:一组服务可以协调工作，组合起来形成定制组合业务需求

服务自治:服务对所封装的逻辑具有控制权

服务无状态:服务将一个活动所需保存的资讯最小化

构建云原生应用程序（SaaS）的12要素

- **1. 基准代码**：一份基准代码，多份部署。基准代码和应用之间总是保持一一对应的关系。所有部署的基准代码相同，但每份部署可以使用其不同的版本。
- **2. 依赖**：显式声明依赖关系。应用程序一定通过依赖清单，确切地声明所有依赖项。
- **3. 配置**：在环境中存储配置。将应用的配置存储于环境变量中。环境变量可以非常方便地在不同的部署间做修改，却不动一行代码。
- **4. 后端服务**：把后端服务当作附加资源。应用不会区别对待本地或第三方服务。对应用程序而言，两种都是附加资源。
- **5. 构建，发布，运行**：严格区分构建，发布，运行这三个步骤。
- **6. 进程**：以一个或多个无状态进程运行应用。应用的进程必须无状态且无共享。
- **7. 端口绑定**：通过端口绑定提供服务。应用完全自我加载而不依赖任何网络服务器就可以创建一个面向网络的服务。
- **8. 并发**：通过进程模型进行扩展。开发人员可以运用这个模型去设计应用架构，将不同工作分配给不同的进程类型。
- **9. 易处理**：快速启动和优雅终止可最大化健壮性。应用的进程是可支配的，意思是说它们可以瞬间开启或停止。
- **10. 开发环境与线上环境等价**：尽可能保持开发、预发布、线上环境相同。应用想要做到持续部署就必须缩小本地与线上差异。
- **11. 日志**：把日志当作事件流。应用本身考虑存储自己的输出流。不应该试图去写或者管理日志文件。
- **12. 管理进程**：后台管理任务当作一次性进程运行。一次性管理进程应该和正常的常驻进程使用同样的环境。

4. 微服务架构风格

（二）微服务（Microservice）架构风格

1. 什么是微服务：**是分布式架构，SOA 的一种扩展**

微服务架构风格是一种将一个单一应用开发为一组小型服务的方法，每个服务运行在自己的进程中，服务间通信采用轻量级通信机制，这些服务围绕业务能力构建并可以通过自动部署机制独立部署。

2. 微服务架构的特点（了解）

a) 通过服务组件化：

- i. 组件是可以独立替换或升级的软件单元。
- ii. 服务是进程外的组件，通过 Web 服务或 RPC 机制通信。
- iii. 微服务架构实现组件化：分解成服务。

- iv. 微服务架构使用服务作为组件原因：服务可以独立部署，产生更明确的发布接口

b) 围绕业务能力组织

- i. 传统软件开发管理聚焦在技术层面，始终存在跨团队成本。
- ii. 微服务开发采用围绕业务能力组织，实现在对应领域内宽栈实现，其团队都是跨职能的。
- iii. 微服务采用产品开发模式，而非项目模式，开发团队负责软件的整个产品流程，持续关注软件如何帮助用户提升业务能力，实现价值交付。

- c) **内聚和解耦**：系统被划分为分离的服务时，可以利用领域驱动设计的历年，将复杂于划分为多个限界上下文（**Bounded Context**）并映射它们之间的关系来澄清和强化分离实现解耦。

d) 去中心化：

- i. 去中心化治理：微服务有自己的技术栈选择，只需要约定接口；运维只需要知道微服务的部署规范。
- ii. 去中心化数据存储：每个服务管理自己的数据库等等
- iii. 去中心化数据管理：服务间无事务协作（权衡业务损失和修复代价）

- e) **基础设施自动化**：云原生等降低构建、部署和运维的成本。

f) 服务设计：

- i. 高可用性：应对服务失效、单独的服务的监控、日志记录等。
- ii. 变更与演化：细粒度的频繁快速发布、根据变更频率进行服务分离。

3.3. 微服务架构与传统架构比较

	优势	劣势
单体	• 人所众知：传统工具、应用和脚本都是这种结构 • IDE友好：Eclipse、IntelliJ等开发工具多 • 便于共享：单个归档文件包含所有功能，便于共享 • 易于测试：单体应用部署后，服务或特性即可展现，没有额外的依赖，测试可以立刻开始 • 容易部署：只需将单个归档文件复制到单个目录下	• 不够灵活：任何细修改需要将整个应用重新构建/部署，这降低了团队的灵活性和功能交付频率 • 妨碍持续交付：单体应用比较大时，构建时间比较长，不利于频繁部署，阻碍持续交付。 • 受技术栈限制：必须使用同一语言/工具、存储及消息，无法根据具体的场景做出其它选择 • 技术债务：“不坏不修（Not broken, don't fix）”，系统设计/代码难以修改，耦合性高。
SOA	• 服务重用性：通过编排你基本服务以用于不同的场景 • 易维护性：单个服务的规模变小，维护相对容易 • 高可靠性：使用消息机制及异步机制，提高了可靠性 • 高扩展和可用性：分布式系统的特性 • 软件质量提升：单个服务的复杂度降低 • 平台无关：可以集成不同的系统 • 提升效率：服务重用、降低复杂性，提升了开发效率	• 过分使用ESB：使得系统集成过于复杂 • 使用基于SOAP协议的WS：使得通信的额外开销很大 • 使用形式化的方式管理：增加了服务管理的复杂度 • 需要使用可靠的ESB：初始投资比较高
微服务	• 简单：单个服务简单，只关注于一个业务功能。 • 团队独立性：每个微服务可以由不同的团队独立开发。 • 松耦合：微服务是松散耦合的。 • 平台无关性：微服务可以用不同的语言/工具开发。 • 通信协议轻量级：使用REST或者RPC进行服务间通信	• 运维成本过高 • 分布式系统的复杂性 • 异步，消息与并行方式使得系统的开发门槛增加 • 分布式系统的复杂性也会让系统的测试变得复杂

3. 微服务架构特点（背诵）

服务颗粒化:服务粒度由业务功能决定，服务间尽可能解耦

责任单一化:单一职责原则，服务内尽可能内聚

运行隔离化:服务运行在各自进程中，互不影响

管理自动化:对服务提供自动化部署与监控预警能力，高效管理

4. 微服务架构的核心模式（详细参见 PPT）

核心模式：针对采用微服务系统在特定场景下的特定问题所使用的程序的架构解决方案的集合。

服务注册与发现：服务消费者获取服务提供者的机制，以实现两者间的解耦服务发现组件。（服务启动时注册，服务调用时发现）

API 网关：所有客户端的单一入口点，针对不同客户端提供不同的 API，解决了微服务结构的应用客户端如何访问各项服务。

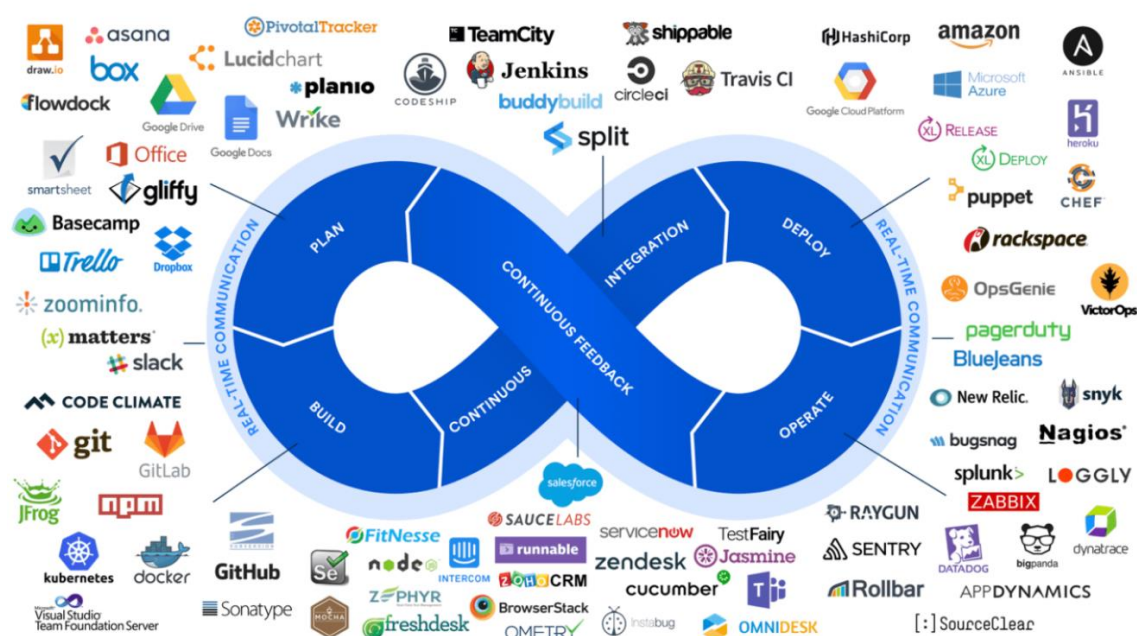
熔断器：避免任何服务或网络出现问题导致的级联故障（可以尝试再次调用）。

闭合状态：请求直接引起方法调用

断开状态：请求立即返回错误响应

半断开状态：允许一定数量的请求调用，如成功则切换到闭合状态，否则切换到断开状态。

5. 微服务架构的实现



6. 微服务架构选型（轻量级）

开发：Spring Boot 封装：Docker 部署：Jenkins 注册 ZooKeeper 调用 Node.js

7. 微服务面临的挑战

运维要求高:微服务数量多，部署与监控要求高

发布复杂度:部署环境多样化，网络性能系统容错、分布式事务等挑战

部署依赖强:服务间相互调用关系复杂，存在部署顺序依赖

通信成本高:跨进程调用比进程内调用消耗更多的资源

附录：（选看）

产品线和产品是什么？

产品线是产品的集合，产品=自定义资产+核心资产。

产品线体系结构与单一体系结构的区分

（产品线是产品的集合）

（单一产品体系结构只有一个产品，软件产品线体系结构有多个产品）

1.软件产品线体系结构能大程度地对现有资源进行重用。软件重用一直是软件工程界不断追求的目标。

2.软件产品线是最大程度的软件重用，可以有效地降低成本、缩短产品面世时间、提高软件质量。

3.单一产品体系结构对体系结构的变化没有说明和限制；必须在软件产品线体系结构中对允许进行的变化进行显式的说明和限定。

4.面向独立软件系统的软件体系结构设计方法并不完全适用于 软件产品线体系结构的设计，因为它们一般没有考虑产品线中不同产品之间的共性和个性问题。

5.产品线体系结构可以使 软件开发组织将总成本均摊到产品线的多个产品的设计开发中，从而充分地降低整体成本、有效地提高软件生产率。

为什么要发展产品线体系结构？（优点？）规模经济、简洁、将项目工作转化为产品功能、提高产品质量、缩短面市时间

产品线优点？

1.大程度地对现有资源进行重用。这是软件工程界不断追求的目标。

2.有效地降低成本、缩短产品面世时间、提高软件质量。

3.降低整体成本、提高软件生产率。

如何达到高可用性？提高代码可修改性。（make code variable）